Studentu rezultatu sistema

v3.0

Generated by Doxygen 1.17.0

1 Directory Hierarchy 1
1.1 Directories . . . 1

2 Hierarchical Index 3
2.1 Class Hierarchy . . . 3

3 Data Structure Index 5
3.1 Data Structures . . . 5

4 File Index 7
4.1 File List . . . 7

5 Directory Documentation 9
5.1 ConsoleApplication1 Directory Reference . . . 9

6 Data Structure Documentation 11
6.1 DuomenuKlaida Struct Reference . . . 11
6.1.1 Detailed Description . . . 11
6.1.2 Constructor & Destructor Documentation . . . 11
6.1.2.1 DuomenuKlaida() . . . 11
6.2 FailoKlaida Struct Reference . . . 12
6.2.1 Detailed Description . . . 12
6.2.2 Constructor & Destructor Documentation . . . 12
6.2.2.1 FailoKlaida() . . . 12
6.3 Studentas Class Reference . . . 12
6.3.1 Detailed Description . . . 13
6.3.2 Constructor & Destructor Documentation . . . 13
6.3.2.1 Studentas() [1/3] . . . 13
6.3.2.2 ∼Studentas() . . . 13
6.3.2.3 Studentas() [2/3] . . . 13
6.3.2.4 Studentas() [3/3] . . . 14
6.3.3 Member Function Documentation . . . 14
6.3.3.1 operator=() [1/2] . . . 14
6.3.3.2 operator=() [2/2] . . . 14
6.3.4 Field Documentation . . . 14
6.3.4.1 egzaminas . . . 14
6.3.4.2 n . . . 14
6.3.4.3 nd . . . 14
6.4 Vector< T > Class Template Reference . . . 15
6.4.1 Detailed Description . . . 17
6.4.2 Member Typedef Documentation . . . 17
6.4.2.1 const_iterator . . . 17
6.4.2.2 iterator . . . 17
6.4.3 Constructor & Destructor Documentation . . . 18

6.4.3.1 Vector() [1/5] . . . 18
6.4.3.2 Vector() [2/5] . . . 18
6.4.3.3 Vector() [3/5] . . . 18
6.4.3.4 ~Vector() . . . 18
6.4.3.5 Vector() [4/5] . . . 19
6.4.3.6 Vector() [5/5] . . . 19
6.4.4 Member Function Documentation . . . 19
6.4.4.1 at() [1/2] . . . 19
6.4.4.2 at() [2/2] . . . 19
6.4.4.3 back() [1/2] . . . 19
6.4.4.4 back() [2/2] . . . 20
6.4.4.5 begin() [1/2] . . . 20
6.4.4.6 begin() [2/2] . . . 20
6.4.4.7 capacity() . . . 20
6.4.4.8 cbegin() . . . 20
6.4.4.9 cend() . . . 20
6.4.4.10 clear() . . . 21
6.4.4.11 data() [1/2] . . . 21
6.4.4.12 data() [2/2] . . . 21
6.4.4.13 emplace() . . . 21
6.4.4.14 emplace_back() . . . 21
6.4.4.15 empty() . . . 22
6.4.4.16 end() [1/2] . . . 22
6.4.4.17 end() [2/2] . . . 22
6.4.4.18 erase() . . . 22
6.4.4.19 front() [1/2] . . . 22
6.4.4.20 front() [2/2] . . . 22
6.4.4.21 insert() . . . 23
6.4.4.22 internal_grow() . . . 23
6.4.4.23 max_size() . . . 23
6.4.4.24 operator"!=() . . . 23
6.4.4.25 operator<() . . . 24
6.4.4.26 operator<=() . . . 24
6.4.4.27 operator=() [1/2] . . . 24
6.4.4.28 operator=() [2/2] . . . 24
6.4.4.29 operator==() . . . 24
6.4.4.30 operator>() . . . 25
6.4.4.31 operator>=() . . . 25
6.4.4.32 operator[]() [1/2] . . . 25
6.4.4.33 operator[]() [2/2] . . . 25
6.4.4.34 pop_back() . . . 25
6.4.4.35 push_back() [1/2] . . . 25

6.4.4.36 push_back() [2/2] . . . 26
6.4.4.37 realloc_count() . . . 26
6.4.4.38 reserve() . . . 26
6.4.4.39 resize() . . . 26
6.4.4.40 shrink_to_fit() . . . 26
6.4.4.41 size() . . . 27
6.4.4.42 swap() . . . 27
6.4.5 Field Documentation . . . 27
6.4.5.1 capacity_ . . . 27
6.4.5.2 data_ . . . 27
6.4.5.3 realloc_count_ . . . 27
6.4.5.4 size_ . . . 27
6.5 Zmogus Class Reference . . . 28
6.5.1 Detailed Description . . . 28
6.5.2 Constructor & Destructor Documentation . . . 28
6.5.2.1 Zmogus() . . . 28
6.5.2.2 ∼Zmogus() . . . 28
6.5.3 Field Documentation . . . 28
6.5.3.1 pavarde . . . 28
6.5.3.2 vardas . . . 28

7 File Documentation **29**

7.1 ConsoleApplication1/exceptions.h File Reference . . . 29
7.2 exceptions.h . . . 29
7.3 ConsoleApplication1/main.cpp File Reference . . . 29
7.3.1 Function Documentation . . . 30
7.3.1.1 main() . . . 30
7.4 main.cpp . . . 30
7.5 ConsoleApplication1/skaiciavimas.cpp File Reference . . . 32
7.5.1 Function Documentation . . . 32
7.5.1.1 skaiciuotiGalutini() . . . 32
7.5.1.2 skaiciuotiMediana() . . . 33
7.5.1.3 skaiciuotiVidurki() . . . 33
7.6 skaiciavimas.cpp . . . 33
7.7 ConsoleApplication1/skaiciavimas.h File Reference . . . 33
7.7.1 Function Documentation . . . 34
7.7.1.1 skaiciuotiGalutini() . . . 34
7.7.1.2 skaiciuotiMediana() . . . 34
7.7.1.3 skaiciuotiVidurki() . . . 34
7.8 skaiciavimas.h . . . 34
7.9 ConsoleApplication1/studentas.cpp File Reference . . . 34
7.9.1 Function Documentation . . . 35

7.9.1.1 operator<<() . . . 35
7.9.1.2 operator>>() . . . 35
7.10 studentas.cpp . . . 35
7.11 ConsoleApplication1/studentas.h File Reference . . . 36
7.11.1 Function Documentation . . . 37
7.11.1.1 operator<<() . . . 37
7.11.1.2 operator>>() . . . 37
7.12 studentas.h . . . 37
7.13 ConsoleApplication1/studentas_utils.cpp File Reference . . . 37
7.13.1 Function Documentation . . . 38
7.13.1.1 generuotiPazymius() . . . 38
7.13.1.2 generuotiVarda() . . . 38
7.13.1.3 pasirinktiRusiavima() . . . 38
7.13.1.4 rusiuotiStudentus() . . . 38
7.14 studentas_utils.cpp . . . 39
7.15 ConsoleApplication1/studentas_utils.h File Reference . . . 40
7.15.1 Function Documentation . . . 40
7.15.1.1 generuotiPazymius() . . . 40
7.15.1.2 generuotiVarda() . . . 40
7.15.1.3 pasirinktiRusiavima() . . . 40
7.15.1.4 rusiuotiStudentus() . . . 40
7.16 studentas_utils.h . . . 41
7.17 ConsoleApplication1/studentu_io.cpp File Reference . . . 41
7.17.1 Function Documentation . . . 41
7.17.1.1 a() . . . 41
7.17.1.2 e() [1/4] . . . 42
7.17.1.3 e() [2/4] . . . 42
7.17.1.4 e() [3/4] . . . 42
7.17.1.5 e() [4/4] . . . 42
7.17.1.6 f() [1/2] . . . 42
7.17.1.7 f() [2/2] . . . 42
7.17.1.8 m() . . . 42
7.17.1.9 s() . . . 43
7.17.2 Variable Documentation . . . 43
7.17.2.1 d . . . 43
7.17.2.2 i . . . 43
7.17.2.3 m . . . 43
7.17.2.4 n . . . 43
7.17.2.5 t . . . 43
7.18 studentu_io.cpp . . . 44
7.19 ConsoleApplication1/studentu_io.h File Reference . . . 46
7.19.1 Function Documentation . . . 47

7.19.1.1 a() . . . 47
7.19.1.2 s() . . . 47
7.19.2 Variable Documentation . . . 47
7.19.2.1 d . . . 47
7.19.2.2 n . . . 47
7.20 studentu_io.h . . . 47
7.21 ConsoleApplication1/testavimas.cpp File Reference . . . 48
7.21.1 Function Documentation . . . 48
7.21.1.1 generuotiFaila() . . . 48
7.21.1.2 isvestiKategorijaIFaila() . . . 48
7.21.1.3 nuskaitytiIsFailo() . . . 49
7.21.1.4 testuotiKlase() . . . 49
7.21.1.5 tyrimas1_failuKurimas() . . . 49
7.21.1.6 tyrimasKonteineriu() . . . 49
7.21.1.7 tyrimasStrategiju() . . . 49
7.21.1.8 tyrimasVectorPushBack() . . . 49
7.21.1.9 tyrimasVectorReallocations() . . . 49
7.21.1.10 tyrimasVectorStudentai() . . . 50
7.22 testavimas.cpp . . . 50
7.23 ConsoleApplication1/testavimas.h File Reference . . . 58
7.23.1 Function Documentation . . . 59
7.23.1.1 apskaiciuotiGalutiniBala() . . . 59
7.23.1.2 benchmarkKonteineris() . . . 59
7.23.1.3 benchmarkStrategijos() . . . 59
7.23.1.4 generuotiFaila() . . . 59
7.23.1.5 isvestiKategorijaIFaila() . . . 59
7.23.1.6 nuskaitytiIsFailo() . . . 60
7.23.1.7 nuskaitytiIsFailoT() . . . 60
7.23.1.8 rusiuotiPagalGalutini() . . . 60
7.23.1.9 strategija1() . . . 60
7.23.1.10 strategija2() . . . 60
7.23.1.11 strategija3() . . . 61
7.23.1.12 testuotiKlase() . . . 61
7.23.1.13 tyrimas1_failuKurimas() . . . 61
7.23.1.14 tyrimasKonteineriu() . . . 61
7.23.1.15 tyrimasStrategiju() . . . 61
7.23.1.16 tyrimasVectorPushBack() . . . 61
7.23.1.17 tyrimasVectorReallocations() . . . 61
7.23.1.18 tyrimasVectorStudentai() . . . 62
7.24 testavimas.h . . . 62
7.25 ConsoleApplication1/vector_stl.h File Reference . . . 64
7.25.1 Detailed Description . . . 65

7.26 vector_stl.h 65
7.27 ConsoleApplication1/zmogus.cpp File Reference 69
7.28 zmogus.cpp 69
7.29 ConsoleApplication1/zmogus.h File Reference 69
7.30 zmogus.h 70

Index **71**

Chapter 1

Directory Hierarchy

1.1 Directories

ConsoleApplication1 9
- exceptions.h 29
- main.cpp 29
- skaiciavimas.cpp 32
- skaiciavimas.h 33
- studentas.cpp 34
- studentas.h 36
- studentas_utils.cpp 37
- studentas_utils.h 40
- studentu_io.cpp 41
- studentu_io.h 46
- testavimas.cpp 48
- testavimas.h 58
- vector_stl.h 64
- zmogus.cpp 69
- zmogus.h 69

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

std::runtime_error
- DuomenuKlaida 11
- FailoKlaida 12

Vector< T > 15

Zmogus 28
- Studentas 12

Chapter 3

Data Structure Index

3.1 Data Structures

Here are the data structures with brief descriptions:

DuomenuKlaida 11
FailoKlaida 12
Studentas
    Studento duomenu klase, paveldinti is Zmogus 12
Vector< T >
    Dinaminis masyvas, atkartojantis std::vector elgesi 15
Zmogus 28

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

ConsoleApplication1/exceptions.h . . . 29
ConsoleApplication1/main.cpp . . . 29
ConsoleApplication1/skaiciavimas.cpp . . . 32
ConsoleApplication1/skaiciavimas.h . . . 33
ConsoleApplication1/studentas.cpp . . . 34
ConsoleApplication1/studentas.h . . . 36
ConsoleApplication1/studentas_utils.cpp . . . 37
ConsoleApplication1/studentas_utils.h . . . 40
ConsoleApplication1/studentu_io.cpp . . . 41
ConsoleApplication1/studentu_io.h . . . 46
ConsoleApplication1/testavimas.cpp . . . 48
ConsoleApplication1/testavimas.h . . . 58
ConsoleApplication1/vector_stl.h
Sablono klase Vector<T>, atkartojanti std::vector funkcionaluma . . . 64
ConsoleApplication1/zmogus.cpp . . . 69
ConsoleApplication1/zmogus.h . . . 69

Chapter 5

Directory Documentation

5.1 ConsoleApplication1 Directory Reference

Files

- file exceptions.h
- file main.cpp
- file skaiciavimas.cpp
- file skaiciavimas.h
- file studentas.cpp
- file studentas.h
- file studentas_utils.cpp
- file studentas_utils.h
- file studentu_io.cpp
- file studentu_io.h
- file testavimas.cpp
- file testavimas.h
- file vector_stl.h

  Sablono klase Vector<T>, atkartojanti std::vector funkcionaluma.
- file zmogus.cpp
- file zmogus.h

Chapter 6

Data Structure Documentation

6.1 DuomenuKlaida Struct Reference

`#include <exceptions.h>`

Inheritance diagram for DuomenuKlaida:

Collaboration diagram for DuomenuKlaida:

Public Member Functions

- DuomenuKlaida (const std::string &msg)

6.1.1 Detailed Description

Definition at line 12 of file exceptions.h.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 DuomenuKlaida()

```
DuomenuKlaida::DuomenuKlaida (
            const std::string & msg)  [inline], [explicit]
```

Definition at line 14 of file exceptions.h.

The documentation for this struct was generated from the following file:

- ConsoleApplication1/exceptions.h

6.2 FailoKlaida Struct Reference

`#include <exceptions.h>`

Inheritance diagram for FailoKlaida:

Collaboration diagram for FailoKlaida:

Public Member Functions

- FailoKlaida (const std::string &msg)

6.2.1 Detailed Description

Definition at line 7 of file exceptions.h.

6.2.2 Constructor & Destructor Documentation

6.2.2.1 FailoKlaida()

```
FailoKlaida::FailoKlaida (
            const std::string & msg)  [inline], [explicit]
```

Definition at line 9 of file exceptions.h.

The documentation for this struct was generated from the following file:

- ConsoleApplication1/exceptions.h

6.3 Studentas Class Reference

Studento duomenu klase, paveldinti is Zmogus.

`#include <studentas.h>`

Inheritance diagram for Studentas:

Collaboration diagram for Studentas:

Public Member Functions

- Studentas ()

  Default konstruktorius. Visi laukai nustatomi i 0/"".
- ~Studentas () noexcept override
- Studentas (const Studentas &kitas)

  Kopijavimo konstruktorius.
- Studentas & operator= (const Studentas &kitas)
- Studentas (Studentas &&kitas) noexcept

  Perkelimo konstruktorius. Saltinis paliekamas tuscias.
- Studentas & operator= (Studentas &&kitas) noexcept

Public Member Functions inherited from Zmogus

- Zmogus ()
- virtual ~Zmogus ()=0

Data Fields

- std::vector< int > nd
- int n
- int egzaminas

Data Fields inherited from Zmogus

- std::string vardas
- std::string pavarde

6.3.1 Detailed Description

Studento duomenu klase, paveldinti is Zmogus.

Realizuoja 5-iu metodu taisykle (Rule of Five): kopijavimo/perkelimo konstruktorius, kopijavimo/perkelimo priskyrimo operatorius ir destruktorius.

Definition at line 14 of file studentas.h.

6.3.2 Constructor & Destructor Documentation

6.3.2.1 Studentas() [1/3]

```
Studentas::Studentas ()
```

Default konstruktorius. Visi laukai nustatomi i 0/"".

Definition at line 4 of file studentas.cpp.

6.3.2.2 ~Studentas()

```
Studentas::~Studentas ()  [override], [noexcept]
```

Definition at line 9 of file studentas.cpp.

6.3.2.3 Studentas() [2/3]

```
Studentas::Studentas (
            const Studentas & kitas)
```

Kopijavimo konstruktorius.

Definition at line 15 of file studentas.cpp.

6.3.2.4 Studentas() [3/3]

```
Studentas::Studentas (
            Studentas && kitas)  [noexcept]
```

Perkelimo konstruktorius. Saltinis paliekamas tuscias.

Definition at line 37 of file studentas.cpp.

6.3.3 Member Function Documentation

6.3.3.1 operator=() [1/2]

```
Studentas & Studentas::operator= (
            const Studentas & kitas)
```

Definition at line 23 of file studentas.cpp.

6.3.3.2 operator=() [2/2]

```
Studentas & Studentas::operator= (
            Studentas && kitas)  [noexcept]
```

Definition at line 49 of file studentas.cpp.

6.3.4 Field Documentation

6.3.4.1 egzaminas

```
int Studentas::egzaminas
```

Definition at line 19 of file studentas.h.

6.3.4.2 n

```
int Studentas::n
```

Definition at line 18 of file studentas.h.

6.3.4.3 nd

```
std::vector<int> Studentas::nd
```

Definition at line 17 of file studentas.h.

The documentation for this class was generated from the following files:

- ConsoleApplication1/studentas.h
- ConsoleApplication1/studentas.cpp

6.4 Vector< T > Class Template Reference

Dinaminis masyvas, atkartojantis std::vector elgesi.

`#include <vector_stl.h>`

Collaboration diagram for Vector< T >:

Public Types

- using iterator = T*
- using const_iterator = const T*

Public Member Functions

- Vector ()

  Default konstruktorius - tuscias vektorius.
- Vector (size_t count, const T &value=T())

  Konstruktorius su pradiniu dydziu.
- Vector (std::initializer_list< T > il)

  Konstruktorius is initializer_list (Vector<int> v = {1,2,3}).
- ~Vector ()

  Destruktorius.
- Vector (const Vector &other)

  Kopijavimo konstruktorius.
- Vector & operator= (const Vector &other)

  Kopijavimo priskyrimas.
- Vector (Vector &&other) noexcept

  Perkelimo konstruktorius.
- Vector & operator= (Vector &&other) noexcept

  Perkelimo priskyrimas.
- bool operator== (const Vector &other) const

  Lyginimo operatorius == (elementu palyginimas).
- bool operator< (const Vector &other) const

  Lyginimo operatorius < (leksikografinis).
- bool operator!= (const Vector &other) const
- bool operator<= (const Vector &other) const
- bool operator> (const Vector &other) const
- bool operator>= (const Vector &other) const
- T & operator[ ] (size_t index)

  Elemento prieiga be ribu tikrinimo.
- const T & operator[ ] (size_t index) const
- T & at (size_t index)

  Elemento prieiga su ribu tikrinimu (gali mesti std::out_of_range).
- const T & at (size_t index) const
- T & front ()
- const T & front () const
- T & back ()
- const T & back () const
- T * data ()
- const T * data () const

- bool empty () const

  Tikrina ar vektorius tuscias.
- size_t size () const

  Grazina elementu skaiciu.
- size_t capacity () const

  Grazina capacity (kiek elementu telpa be reallocation).
- size_t max_size () const

  Maksimalus galimas elementu skaicius.
- size_t realloc_count () const

  Grazina kiek kartu ivyko vidinis perskirstymas.
- void reserve (size_t new_cap)

  Rezervuoja vietos bent new_cap elementu, jei dar nera.
- void shrink_to_fit ()

  Sumazina capacity iki size (jei reikia).
- iterator begin ()
- iterator end ()
- const_iterator begin () const
- const_iterator end () const
- const_iterator cbegin () const
- const_iterator cend () const
- void push_back (const T &value)

  Prideda elementa i pabaiga (kopija).
- void push_back (T &&value)

  Prideda elementa i pabaiga (perkelimas).
- void pop_back ()

  Pasalina paskutini elementa.
- void clear ()

  Istusina vektoriu (size_ = 0, capacity nepasikeicia).
- iterator insert (const_iterator pos, const T &value)

  Iterpia elementa pries pos pozicija.
- iterator erase (const_iterator pos)

  Pasalina elementa pos pozicijoje.
- template<typename ... Args>
  iterator emplace (const_iterator pos, Args &&... args)

  Iterpia konstrukuojant elementa pries pos pozicija (perfect forwarding).
- template<typename ... Args>
  void emplace_back (Args &&... args)

  Konstrukuoja elementa pabaigoje (perfect forwarding).
- void resize (size_t count, const T &value=T())

  Pakeicia dydi i count, ipildo value, jei reikia padidinti.
- void swap (Vector &other) noexcept

  Sukeicia dvieju vektoriu turini.

Private Member Functions

- void internal_grow (size_t wanted=0)

  Vidinis buferiu perskirstymas.

Private Attributes

- T * data_ = nullptr
- size_t size_ = 0
- size_t capacity_ = 0
- size_t realloc_count_ = 0

6.4.1 Detailed Description

template<typename T>
class Vector< T >

Dinaminis masyvas, atkartojantis std::vector elgesi.

Klase realizuoja: Rule of Five (kopijavimo / perkelimo ctor + operator=), elementu prieigos metodus (operator[], at, front, back, data), dydzio valdyma (size, capacity, empty, reserve, resize, shrink_to_fit, clear), keitimo metodus (push_back, pop_back, insert, erase, emplace, emplace_back, swap), iteratorius (begin, end, cbegin, cend) ir lyginimo operatorius.

Papildomai realizuotas reallocation skaitliukas - realloc_count() grazina kiek kartu vidiniu buferiu perskirstymas (capacity perskaiciavimas) buvo atliktas.

Template Parameters

T	elemento tipas.

Definition at line 30 of file vector_stl.h.

6.4.2 Member Typedef Documentation

6.4.2.1 const_iterator

```
template<typename T>
using Vector< T >::const_iterator = const T*
```

Definition at line 39 of file vector_stl.h.

6.4.2.2 iterator

```
template<typename T>
using Vector< T >::iterator = T*
```

Definition at line 38 of file vector_stl.h.

6.4.3 Constructor & Destructor Documentation

6.4.3.1 Vector() [1/5]

```
template<typename T>
Vector< T >::Vector ()  [inline]
```

Default konstruktorius - tuscias vektorius.

Definition at line 43 of file vector_stl.h.

6.4.3.2 Vector() [2/5]

```
template<typename T>
Vector< T >::Vector (
            size_t count,
            const T & value = T( ))  [inline], [explicit]
```

Konstruktorius su pradiniu dydziu.

Parameters

count	elementu skaicius.
value	pradine reiksme (default = T()).

Definition at line 50 of file vector_stl.h.

6.4.3.3 Vector() [3/5]

```
template<typename T>
Vector< T >::Vector (
            std::initializer_list< T > il)  [inline]
```

Konstruktorius is initializer_list (Vector<int> v = {1,2,3}).

Definition at line 64 of file vector_stl.h.

6.4.3.4 ~Vector()

```
template<typename T>
Vector< T >::~Vector ()  [inline]
```

Destruktorius.

Definition at line 73 of file vector_stl.h.

6.4.3.5 Vector() [4/5]

```
template<typename T>
Vector< T >::Vector (
            const Vector< T > & other)  [inline]
```

Kopijavimo konstruktorius.

Definition at line 76 of file vector_stl.h.

6.4.3.6 Vector() [5/5]

```
template<typename T>
Vector< T >::Vector (
            Vector< T > && other)  [inline], [noexcept]
```

Perkelimo konstruktorius.

Definition at line 105 of file vector_stl.h.

6.4.4 Member Function Documentation

6.4.4.1 at() [1/2]

```
template<typename T>
T & Vector< T >::at (
            size_t index)  [inline]
```

Elemento prieiga su ribu tikrinimu (gali mesti std::out_of_range).

Definition at line 191 of file vector_stl.h.

6.4.4.2 at() [2/2]

```
template<typename T>
const T & Vector< T >::at (
            size_t index) const  [inline]
```

Definition at line 199 of file vector_stl.h.

6.4.4.3 back() [1/2]

```
template<typename T>
T & Vector< T >::back ()  [inline]
```

Definition at line 210 of file vector_stl.h.

6.4.4.4 back() [2/2]

```
template<typename T>
const T & Vector< T >::back () const  [inline]
```

Definition at line 211 of file vector_stl.h.

6.4.4.5 begin() [1/2]

```
template<typename T>
iterator Vector< T >::begin ()  [inline]
```

Definition at line 271 of file vector_stl.h.

6.4.4.6 begin() [2/2]

```
template<typename T>
const_iterator Vector< T >::begin () const  [inline]
```

Definition at line 274 of file vector_stl.h.

6.4.4.7 capacity()

```
template<typename T>
size_t Vector< T >::capacity () const  [inline]
```

Grazina capacity (kiek elementu telpa be reallocation).

Definition at line 223 of file vector_stl.h.

6.4.4.8 cbegin()

```
template<typename T>
const_iterator Vector< T >::cbegin () const  [inline]
```

Definition at line 277 of file vector_stl.h.

6.4.4.9 cend()

```
template<typename T>
const_iterator Vector< T >::cend () const  [inline]
```

Definition at line 278 of file vector_stl.h.

6.4.4.10 clear()

```
template<typename T>
void Vector< T >::clear ()  [inline]
```

Istusina vektoriu (size_ = 0, capacity nepasikeicia).

Definition at line 308 of file vector_stl.h.

6.4.4.11 data() [1/2]

```
template<typename T>
T * Vector< T >::data ()  [inline]
```

Definition at line 213 of file vector_stl.h.

6.4.4.12 data() [2/2]

```
template<typename T>
const T * Vector< T >::data () const  [inline]
```

Definition at line 214 of file vector_stl.h.

6.4.4.13 emplace()

```
template<typename T>
template<typename ...  Args>
iterator Vector< T >::emplace (
            const_iterator pos,
            Args &&...  args)  [inline]
```

Iterpia konstrukuojant elementa pries pos pozicija (perfect forwarding).

Definition at line 352 of file vector_stl.h.

6.4.4.14 emplace_back()

```
template<typename T>
template<typename ...  Args>
void Vector< T >::emplace_back (
            Args &&...  args)  [inline]
```

Konstrukuoja elementa pabaigoje (perfect forwarding).

Definition at line 371 of file vector_stl.h.

6.4.4.15 empty()

```
template<typename T>
bool Vector< T >::empty () const  [inline]
```

Tikrina ar vektorius tuscias.

Definition at line 217 of file vector_stl.h.

6.4.4.16 end() [1/2]

```
template<typename T>
iterator Vector< T >::end ()  [inline]
```

Definition at line 272 of file vector_stl.h.

6.4.4.17 end() [2/2]

```
template<typename T>
const_iterator Vector< T >::end () const  [inline]
```

Definition at line 275 of file vector_stl.h.

6.4.4.18 erase()

```
template<typename T>
iterator Vector< T >::erase (
            const_iterator pos)  [inline]
```

Pasalina elementa pos pozicijoje.

Returns

iteratorius i kita elementa po pasalintojo.

Definition at line 335 of file vector_stl.h.

6.4.4.19 front() [1/2]

```
template<typename T>
T & Vector< T >::front ()  [inline]
```

Definition at line 207 of file vector_stl.h.

6.4.4.20 front() [2/2]

```
template<typename T>
const T & Vector< T >::front () const  [inline]
```

Definition at line 208 of file vector_stl.h.

6.4.4.21 insert()

```
template<typename T>
iterator Vector< T >::insert (
            const_iterator pos,
            const T & value)  [inline]
```

Iterpia elementa pries pos pozicija.

Returns

iteratorius i naujai iterpta elementa.

Definition at line 314 of file vector_stl.h.

6.4.4.22 internal_grow()

```
template<typename T>
void Vector< T >::internal_grow (
            size_t wanted = 0)  [inline], [private]
```

Vidinis buferiu perskirstymas.

Parameters

wanted	norimas naujas capacity (0 = automatinis x2 augimas).

Definition at line 168 of file vector_stl.h.

6.4.4.23 max_size()

```
template<typename T>
size_t Vector< T >::max_size () const  [inline]
```

Maksimalus galimas elementu skaicius.

Definition at line 226 of file vector_stl.h.

6.4.4.24 operator"!=()

```
template<typename T>
bool Vector< T >::operator!= (
            const Vector< T > & other) const  [inline]
```

Definition at line 158 of file vector_stl.h.

6.4.4.25 operator<()

```
template<typename T>
bool Vector< T >::operator< (
            const Vector< T > & other) const  [inline]
```

Lyginimo operatorius < (leksikografinis).

Definition at line 148 of file vector_stl.h.

6.4.4.26 operator<=()

```
template<typename T>
bool Vector< T >::operator<= (
            const Vector< T > & other) const  [inline]
```

Definition at line 159 of file vector_stl.h.

6.4.4.27 operator=() [1/2]

```
template<typename T>
Vector & Vector< T >::operator= (
            const Vector< T > & other)  [inline]
```

Kopijavimo priskyrimas.

Definition at line 88 of file vector_stl.h.

6.4.4.28 operator=() [2/2]

```
template<typename T>
Vector & Vector< T >::operator= (
            Vector< T > && other)  [inline], [noexcept]
```

Perkelimo priskyrimas.

Definition at line 114 of file vector_stl.h.

6.4.4.29 operator==()

```
template<typename T>
bool Vector< T >::operator== (
            const Vector< T > & other) const  [inline]
```

Lyginimo operatorius == (elementu palyginimas).

Definition at line 135 of file vector_stl.h.

6.4.4.30 operator>()

```
template<typename T>
bool Vector< T >::operator> (
            const Vector< T > & other) const  [inline]
```

Definition at line 160 of file vector_stl.h.

6.4.4.31 operator>=()

```
template<typename T>
bool Vector< T >::operator>= (
            const Vector< T > & other) const  [inline]
```

Definition at line 161 of file vector_stl.h.

6.4.4.32 operator[]() [1/2]

```
template<typename T>
T & Vector< T >::operator[] (
            size_t index)  [inline]
```

Elemento prieiga be ribu tikrinimo.

Definition at line 187 of file vector_stl.h.

6.4.4.33 operator[]() [2/2]

```
template<typename T>
const T & Vector< T >::operator[] (
            size_t index) const  [inline]
```

Definition at line 188 of file vector_stl.h.

6.4.4.34 pop_back()

```
template<typename T>
void Vector< T >::pop_back ()  [inline]
```

Pasalina paskutini elementa.

Definition at line 299 of file vector_stl.h.

6.4.4.35 push_back() [1/2]

```
template<typename T>
void Vector< T >::push_back (
            const T & value)  [inline]
```

Prideda elementa i pabaiga (kopija).

Definition at line 282 of file vector_stl.h.

6.4.4.36 push_back() [2/2]

```
template<typename T>
void Vector< T >::push_back (
            T && value)  [inline]
```

Prideda elementa i pabaiga (perkelimas).

Definition at line 291 of file vector_stl.h.

6.4.4.37 realloc_count()

```
template<typename T>
size_t Vector< T >::realloc_count () const  [inline]
```

Grazina kiek kartu ivyko vidinis perskirstymas.

Definition at line 229 of file vector_stl.h.

6.4.4.38 reserve()

```
template<typename T>
void Vector< T >::reserve (
            size_t new_cap)  [inline]
```

Rezervuoja vietos bent new_cap elementu, jei dar nera.

Parameters

new_cap	norima capacity reiksme.

Definition at line 235 of file vector_stl.h.

6.4.4.39 resize()

```
template<typename T>
void Vector< T >::resize (
            size_t count,
            const T & value = T( ))  [inline]
```

Pakeicia dydi i count, ipildo value, jei reikia padidinti.

Definition at line 383 of file vector_stl.h.

6.4.4.40 shrink_to_fit()

```
template<typename T>
void Vector< T >::shrink_to_fit ()  [inline]
```

Sumazina capacity iki size (jei reikia).

Definition at line 244 of file vector_stl.h.

6.4.4.41 size()

```
template<typename T>
size_t Vector< T >::size () const  [inline]
```

Grazina elementu skaiciu.

Definition at line 220 of file vector_stl.h.

6.4.4.42 swap()

```
template<typename T>
void Vector< T >::swap (
            Vector< T > & other)  [inline], [noexcept]
```

Sukeicia dvieju vektoriu turini.

Definition at line 403 of file vector_stl.h.

6.4.5 Field Documentation

6.4.5.1 capacity_

```
template<typename T>
size_t Vector< T >::capacity_ = 0  [private]
```

Definition at line 34 of file vector_stl.h.

6.4.5.2 data_

```
template<typename T>
T* Vector< T >::data_ = nullptr  [private]
```

Definition at line 32 of file vector_stl.h.

6.4.5.3 realloc_count_

```
template<typename T>
size_t Vector< T >::realloc_count_ = 0  [private]
```

Definition at line 35 of file vector_stl.h.

6.4.5.4 size_

```
template<typename T>
size_t Vector< T >::size_ = 0  [private]
```

Definition at line 33 of file vector_stl.h.

The documentation for this class was generated from the following file:

- ConsoleApplication1/vector_stl.h

6.5 Zmogus Class Reference

`#include <zmogus.h>`

Inheritance diagram for Zmogus:

Collaboration diagram for Zmogus:

Public Member Functions

- Zmogus ()
- virtual ~Zmogus ()=0

Data Fields

- std::string vardas
- std::string pavarde

6.5.1 Detailed Description

Definition at line 7 of file zmogus.h.

6.5.2 Constructor & Destructor Documentation

6.5.2.1 Zmogus()

```
Zmogus::Zmogus ()
```

Definition at line 3 of file zmogus.cpp.

6.5.2.2 ~Zmogus()

```
Zmogus::~Zmogus ()  [pure virtual]
```

Definition at line 7 of file zmogus.cpp.

6.5.3 Field Documentation

6.5.3.1 pavarde

```
std::string Zmogus::pavarde
```

Definition at line 11 of file zmogus.h.

6.5.3.2 vardas

```
std::string Zmogus::vardas
```

Definition at line 10 of file zmogus.h.

The documentation for this class was generated from the following files:

- ConsoleApplication1/zmogus.h
- ConsoleApplication1/zmogus.cpp

Chapter 7

File Documentation

7.1 ConsoleApplication1/exceptions.h File Reference

```
#include <stdexcept>
#include <string>
```

Include dependency graph for exceptions.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct FailoKlaida
- struct DuomenuKlaida

7.2 exceptions.h

Go to the documentation of this file.

```
00001 #ifndef EXCEPTIONS_H
00002 #define EXCEPTIONS_H
00003
00004 #include <stdexcept>
00005 #include <string>
00006
00007 struct FailoKlaida : public std::runtime_error
00008 {
00009     explicit FailoKlaida( const std::string& msg ) : std::runtime_error( msg ) {}
00010 };
00011
00012 struct DuomenuKlaida : public std::runtime_error
00013 {
00014     explicit DuomenuKlaida( const std::string& msg ) : std::runtime_error( msg ) {}
00015 };
00016
00017 #endif
```

7.3 ConsoleApplication1/main.cpp File Reference

```
#include "studentu_io.h"
#include "studentas_utils.h"
#include "exceptions.h"
#include "testavimas.h"
#include <iostream>
#include <cstdlib>
#include <ctime>
```

Include dependency graph for main.cpp:

Functions

- int main ()

7.3.1 Function Documentation

7.3.1.1 main()

```
int main ()
```

Definition at line 10 of file main.cpp.

7.4 main.cpp

Go to the documentation of this file.

```
00001 #include "studentu_io.h"
00002 #include "studentas_utils.h"
00003 #include "exceptions.h"
00004 #include "testavimas.h"
00005 #include <iostream>
00006 #include <cstdlib>
00007 #include <ctime>
00008
00009
00010 int main( )
00011 {
00012     std::srand( static_cast<unsigned>( std::time( nullptr ) ) );
00013
00014     int skaiciavimas = { };
00015     std::cout « "Galutinio balo skaiciavimas:\n  1 - Vidurkis\n  2 - Mediana\nPasirinkimas: ";
00016
00017     while ( !skaitytiSveika( skaiciavimas, 1, 2 ) ) {
00018         std::cout « "Neteisinga reiksme. Pasirinkite 1 arba 2: ";
00019     }
00020
00021     bool mediana = ( skaiciavimas == 2 );
00022
00023     int m { };
00024     int n { };
00025     int meniu;
00026
00027     do {
00028         std::cout « "\n========== MENIU ==========\n";
00029         std::cout « "  1  - Ivesti duomenis rankiniu budu\n";
00030         std::cout « "  2  - Generuoti tik pazymius\n";
00031         std::cout « "  3  - Generuoti vardus, pavardes ir pazymius\n";
00032         std::cout « "  4  - Nuskaityti studentus is failo\n";
00033         std::cout « "  5  - 1 tyrimas: Failu generavimas\n";
00034         std::cout « "  6  - 2 tyrimas: Konteineriu palyginimas (vector, list, deque)\n";
00035         std::cout « "  7  - 3 tyrimas: Strategiju palyginimas (1, 2, 3 strategijos)\n";
00036         std::cout « "  8  - Testuoti Studentas klase (Rule of Five + I/O operatoriai)\n";
00037         std::cout « "  9  - V3.0 tyrimas: push_back std::vector vs Vector\n";
00038         std::cout « "  10 - V3.0 tyrimas: atminties perskirstymai (100M elementu)\n";
00039         std::cout « "  11 - V3.0 tyrimas: Studentu programa std::vector vs Vector\n";
00040         std::cout « "  12 - Baigti darba\n";
00041         std::cout « "Pasirinkimas: ";
00042
00043         if ( !skaitytiSveika( meniu, 1, 12 ) )
00044         {
00045             std::cout « "Neteisinga reiksme.\n";
00046             continue;
00047         }
00048
00049         try
00050         {
00051             switch ( meniu )
00052             {
00053             case 1:
00054             {
00055                 std::vector<Studentas> studentai = ivestiRankiniu( m, n );
00056
```

```
00057                 if ( m > 0 )
00058                     spausdintiRezultatus( studentai, m, mediana );
00059                 else
00060                     std::cout « "Nera studentu duomenu.\n";
00061                 break;
00062             }
00063
00064             case 2:
00065             {
00066                 std::cout « "Studentu skaicius: ";
00067                 while ( !skaitytiSveika( m, 1, 10000 ) ) {
00068                     std::cout « "Neteisinga reiksme: ";
00069                 }
00070
00071                 std::cout « "Namu darbu skaicius: ";
00072                 while ( !skaitytiSveika( n, 1, 100 ) ) {
00073                     std::cout « "Neteisinga reiksme: ";
00074                 }
00075
00076                 std::vector<Studentas> studentai( m );
00077
00078                 for ( int i = 0; i < m; i++ )
00079                 {
00080                     std::cout « "  Studentas #" « ( i + 1 ) « " vardas: ";
00081                     std::cin » studentai[i].vardas;
00082
00083                     std::cout « "  Studentas #" « ( i + 1 ) « " pavarde: ";
00084                     std::cin » studentai[i].pavarde;
00085
00086                     generuotiPazymius( studentai[i], n );
00087                 }
00088
00089                 spausdintiRezultatus( studentai, m, mediana );
00090                 break;
00091             }
00092
00093             case 3:
00094             {
00095                 std::cout « "Studentu skaicius: ";
00096                 while ( !skaitytiSveika( m, 1, 10000 ) ) {
00097                     std::cout « "Neteisinga reiksme: ";
00098                 }
00099
00100                 std::cout « "Namu darbu skaicius: ";
00101                 while ( !skaitytiSveika( n, 1, 100 ) ) {
00102                     std::cout « "Neteisinga reiksme: ";
00103                 }
00104
00105                 std::vector<Studentas> studentai( m );
00106
00107                 for ( int i = 0; i < m; i++ )
00108                 {
00109                     generuotiVarda( studentai[i], i );
00110                     generuotiPazymius( studentai[i], n );
00111                 }
00112
00113                 spausdintiRezultatus( studentai, m, mediana );
00114                 break;
00115             }
00116
00117             case 4:
00118             {
00119                 std::vector<Studentas> studentai = nuskaitytiStudentus( );
00120
00121                 if ( studentai.size( ) > 0 )
00122                     spausdintiRezultatus( studentai, static_cast<int>( studentai.size( ) ), mediana );
00123                 else
00124                     std::cout « "Nera studentu duomenu.\n";
00125
00126                 break;
00127             }
00128
00129             case 5:
00130             {
00131                 tyrimas1_failuKurimas( );
00132                 break;
00133             }
00134
00135             case 6:
00136             {
00137                 tyrimasKonteineriu( mediana );
00138                 break;
00139             }
00140
00141             case 7:
00142             {
00143                 tyrimasStrategiju( mediana );
```

```
00144                 break;
00145             }
00146
00147             case 8:
00148             {
00149                 testuotiKlase();
00150                 break;
00151             }
00152
00153             case 9:
00154             {
00155                 tyrimasVectorPushBack( );
00156                 break;
00157             }
00158
00159             case 10:
00160             {
00161                 tyrimasVectorReallocations( );
00162                 break;
00163             }
00164
00165             case 11:
00166             {
00167                 tyrimasVectorStudentai( mediana );
00168                 break;
00169             }
00170
00171             case 12:
00172             {
00173                 std::cout « "Programa baigta.\n";
00174                 break;
00175             }
00176             }
00177         }
00178         catch ( const FailoKlaida& e )
00179         {
00180             std::cerr « "Failo klaida: " « e.what( ) « "\n";
00181         }
00182         catch ( const DuomenuKlaida& e )
00183         {
00184             std::cerr « "Duomenu klaida: " « e.what( ) « "\n";
00185         }
00186         catch ( const std::exception& e )
00187         {
00188             std::cerr « "Klaida: " « e.what( ) « "\n";
00189         }
00190
00191     } while ( meniu != 12 );
00192
00193     return 0;
00194 }
```

7.5 ConsoleApplication1/skaiciavimas.cpp File Reference

```
#include "skaiciavimas.h"
#include <algorithm>
```

Include dependency graph for skaiciavimas.cpp:

Functions

- double skaiciuotiVidurki (const std::vector< int > &nd, int n)
- double skaiciuotiMediana (const std::vector< int > &nd, int n)
- double skaiciuotiGalutini (double nd_rezultatas, int egz)

7.5.1 Function Documentation

7.5.1.1 skaiciuotiGalutini()

```
double skaiciuotiGalutini (
            double nd_rezultatas,
            int egz)
```

Definition at line 34 of file skaiciavimas.cpp.

7.5.1.2 skaiciuotiMediana()

```
double skaiciuotiMediana (
            const std::vector< int > & nd,
            int n)
```

Definition at line 16 of file skaiciavimas.cpp.

7.5.1.3 skaiciuotiVidurki()

```
double skaiciuotiVidurki (
            const std::vector< int > & nd,
            int n)
```

Definition at line 4 of file skaiciavimas.cpp.

7.6 skaiciavimas.cpp

Go to the documentation of this file.

```
00001 #include "skaiciavimas.h"
00002 #include <algorithm>
00003
00004 double skaiciuotiVidurki( const std::vector<int>& nd, int n )
00005 {
00006     if ( n == 0 )
00007         return 0.0;
00008
00009     double suma = 0;
00010     for ( int i = 0; i < n; i++ )
00011         suma += nd [ i ];
00012
00013     return suma / n;
00014 }
00015
00016 double skaiciuotiMediana( const std::vector<int>& nd, int n )
00017 {
00018     if ( n == 0 )
00019         return 0.0;
00020
00021     std::vector<int> copy = nd;
00022     std::sort( copy.begin(), copy.end() );
00023
00024     double rezultatas;
00025
00026     if ( n % 2 == 0 )
00027         rezultatas = ( copy [ n / 2 - 1 ] + copy [ n / 2 ] ) / 2.0;
00028     else
00029         rezultatas = copy [ n / 2 ];
00030
00031     return rezultatas;
00032 }
00033
00034 double skaiciuotiGalutini( double nd_rezultatas, int egz )
00035 {
00036     return 0.4 * nd_rezultatas + 0.6 * egz;
00037 }
```

7.7 ConsoleApplication1/skaiciavimas.h File Reference

```
#include <vector>
```

Include dependency graph for skaiciavimas.h: This graph shows which files directly or indirectly include this file:

Functions

- double skaiciuotiVidurki (const std::vector< int > &nd, int n)
- double skaiciuotiMediana (const std::vector< int > &nd, int n)
- double skaiciuotiGalutini (double nd_rezultatas, int egz)

7.7.1 Function Documentation

7.7.1.1 skaiciuotiGalutini()

```
double skaiciuotiGalutini (
            double nd_rezultatas,
            int egz)
```

Definition at line 34 of file skaiciavimas.cpp.

7.7.1.2 skaiciuotiMediana()

```
double skaiciuotiMediana (
            const std::vector< int > & nd,
            int n)
```

Definition at line 16 of file skaiciavimas.cpp.

7.7.1.3 skaiciuotiVidurki()

```
double skaiciuotiVidurki (
            const std::vector< int > & nd,
            int n)
```

Definition at line 4 of file skaiciavimas.cpp.

7.8 skaiciavimas.h

Go to the documentation of this file.

```
00001 #ifndef SKAICIAVIMAS_H
00002 #define SKAICIAVIMAS_H
00003
00004 #include <vector>
00005
00006 double skaiciuotiVidurki( const std::vector<int>& nd, int n );
00007 double skaiciuotiMediana( const std::vector<int>& nd, int n );
00008 double skaiciuotiGalutini( double nd_rezultatas, int egz );
00009
00010 #endif
```

7.9 ConsoleApplication1/studentas.cpp File Reference

```
#include "studentas.h"
#include <utility>
```

Include dependency graph for studentas.cpp:

Functions

- std::ostream & operator<< (std::ostream &out, const Studentas &s)
- std::istream & operator>> (std::istream &in, Studentas &s)

7.9.1 Function Documentation

7.9.1.1 operator<<()

```
std::ostream & operator<< (
            std::ostream & out,
            const Studentas & s)
```

Definition at line 66 of file studentas.cpp.

7.9.1.2 operator>>()

```
std::istream & operator>> (
            std::istream & in,
            Studentas & s)
```

Definition at line 78 of file studentas.cpp.

7.10 studentas.cpp

Go to the documentation of this file.

```
00001 #include "studentas.h"
00002 #include <utility>
00003
00004 Studentas::Studentas()
00005     : Zmogus(), nd(), n( 0 ), egzaminas( 0 )
00006 {
00007 }
00008
00009 Studentas::~Studentas() noexcept
00010 {
00011     n = 0;
00012     egzaminas = 0;
00013 }
00014
00015 Studentas::Studentas( const Studentas& kitas )
00016     : Zmogus( kitas ),
00017       nd( kitas.nd ),
00018       n( kitas.n ),
00019       egzaminas( kitas.egzaminas )
00020 {
00021 }
00022
00023 Studentas& Studentas::operator=( const Studentas& kitas )
00024 {
00025     if ( this == &kitas )
00026         return *this;
00027
00028     vardas    = kitas.vardas;
00029     pavarde   = kitas.pavarde;
00030     nd        = kitas.nd;
00031     n         = kitas.n;
00032     egzaminas = kitas.egzaminas;
00033
00034     return *this;
00035 }
00036
00037 Studentas::Studentas( Studentas&& kitas ) noexcept
```

```
00038     : Zmogus(),
00039       nd( std::move( kitas.nd ) ),
00040       n( kitas.n ),
00041       egzaminas( kitas.egzaminas )
00042 {
00043     vardas          = std::move( kitas.vardas );
00044     pavarde         = std::move( kitas.pavarde );
00045     kitas.n         = 0;
00046     kitas.egzaminas = 0;
00047 }
00048
00049 Studentas& Studentas::operator=( Studentas&& kitas ) noexcept
00050 {
00051     if ( this == &kitas )
00052         return *this;
00053
00054     vardas    = std::move( kitas.vardas );
00055     pavarde   = std::move( kitas.pavarde );
00056     nd        = std::move( kitas.nd );
00057     n         = kitas.n;
00058     egzaminas = kitas.egzaminas;
00059
00060     kitas.n         = 0;
00061     kitas.egzaminas = 0;
00062
00063     return *this;
00064 }
00065
00066 std::ostream& operator«( std::ostream& out, const Studentas& s )
00067 {
00068     out « s.vardas « " " « s.pavarde « " " « s.n;
00069
00070     for ( int i = 0; i < s.n; i++ )
00071         out « " " « s.nd[ i ];
00072
00073     out « " " « s.egzaminas;
00074
00075     return out;
00076 }
00077
00078 std::istream& operator»( std::istream& in, Studentas& s )
00079 {
00080     if ( !( in » s.vardas » s.pavarde » s.n ) )
00081         return in;
00082
00083     s.nd.resize( s.n );
00084
00085     for ( int i = 0; i < s.n; i++ )
00086     {
00087         if ( !( in » s.nd[ i ] ) )
00088             return in;
00089     }
00090
00091     in » s.egzaminas;
00092
00093     return in;
00094 }
```

7.11 ConsoleApplication1/studentas.h File Reference

```
#include "zmogus.h"
#include <vector>
#include <iostream>
```

Include dependency graph for studentas.h: This graph shows which files directly or indirectly include this file:

Data Structures

- class Studentas

  Studento duomenu klase, paveldinti is Zmogus.

Functions

- std::ostream & operator<< (std::ostream &out, const Studentas &s)
- std::istream & operator>> (std::istream &in, Studentas &s)

7.11.1 Function Documentation

7.11.1.1 operator<<()

```
std::ostream & operator<< (
            std::ostream & out,
            const Studentas & s)
```

Definition at line 66 of file studentas.cpp.

7.11.1.2 operator>>()

```
std::istream & operator>> (
            std::istream & in,
            Studentas & s)
```

Definition at line 78 of file studentas.cpp.

7.12 studentas.h

Go to the documentation of this file.

```
00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include "zmogus.h"
00005 #include <vector>
00006 #include <iostream>
00007
00014 class Studentas : public Zmogus
00015 {
00016 public:
00017     std::vector<int> nd;
00018     int n;
00019     int egzaminas;
00020
00022     Studentas();
00023     ~Studentas() noexcept override;
00024
00026     Studentas( const Studentas& kitas );
00027     Studentas& operator=( const Studentas& kitas );
00028
00030     Studentas( Studentas&& kitas ) noexcept;
00031     Studentas& operator=( Studentas&& kitas ) noexcept;
00032 };
00033
00034 std::ostream& operator«( std::ostream& out, const Studentas& s );
00035 std::istream& operator»( std::istream& in, Studentas& s );
00036
00037 #endif
```

7.13 ConsoleApplication1/studentas_utils.cpp File Reference

```
#include "studentas_utils.h"
#include "skaiciavimas.h"
#include "studentu_io.h"
#include <iostream>
#include <algorithm>
#include <cstdlib>
```

Include dependency graph for studentas_utils.cpp:

Functions

- void generuotiPazymius (Studentas &s, int n)
- void generuotiVarda (Studentas &s, int indeksas)
- int pasirinktiRusiavima ()
- void rusiuotiStudentus (std::vector< Studentas > &studentai, int rusiavimas)

7.13.1 Function Documentation

7.13.1.1 generuotiPazymius()

```
void generuotiPazymius (
            Studentas & s,
            int n)
```

Definition at line 8 of file studentas_utils.cpp.

7.13.1.2 generuotiVarda()

```
void generuotiVarda (
            Studentas & s,
            int indeksas)
```

Definition at line 19 of file studentas_utils.cpp.

7.13.1.3 pasirinktiRusiavima()

```
int pasirinktiRusiavima ()
```

Definition at line 29 of file studentas_utils.cpp.

7.13.1.4 rusiuotiStudentus()

```
void rusiuotiStudentus (
            std::vector< Studentas > & studentai,
            int rusiavimas)
```

Definition at line 48 of file studentas_utils.cpp.

7.14 studentas_utils.cpp

Go to the documentation of this file.

```
00001 #include "studentas_utils.h"
00002 #include "skaiciavimas.h"
00003 #include "studentu_io.h"
00004 #include <iostream>
00005 #include <algorithm>
00006 #include <cstdlib>
00007
00008 void generuotiPazymius( Studentas& s, int n )
00009 {
00010     s.n = n;
00011     s.nd.resize( n );
00012
00013     for ( int i = 0; i < n; i++ )
00014         s.nd [ i ] = rand( ) % 10 + 1;
00015
00016     s.egzaminas = rand( ) % 10 + 1;
00017 }
00018
00019 void generuotiVarda( Studentas& s, int indeksas )
00020 {
00021     std::string vardai[ ] = { "Jonas", "Petras", "Ona", "Marta", "Lukas", "Egle", "Tomas", "Inga" };
00022     std::string pavardes[ ] = { "Jonaitis", "Petraitis", "Kazlauskas", "Stankevicious", "Vaitkus",
      "Lukosius" };
00023
00024
00025     s.vardas = vardai [ indeksas % 8 ];
00026     s.pavarde = pavardes [ indeksas % 6 ];
00027 }
00028
00029 int pasirinktiRusiavima( )
00030 {
00031     int pasirinkimas;
00032
00033
00034     std::cout << "\nRusiavimo pasirinkimas:\n";
00035     std::cout << "  1 - Pagal varda\n";
00036     std::cout << "  2 - Pagal pavarde\n";
00037     std::cout << "  3 - Pagal galutini (vidurkis)\n";
00038     std::cout << "  4 - Pagal galutini (mediana)\n";
00039     std::cout << "Pasirinkimas: ";
00040
00041     while ( !skaitytiSveika( pasirinkimas, 1, 4 ) ) {
00042         std::cout << "Neteisinga reiksme. Pasirinkite 1-4: ";
00043     }
00044
00045     return pasirinkimas;
00046 }
00047
00048 void rusiuotiStudentus( std::vector<Studentas>& studentai, int rusiavimas )
00049 {
00050     switch ( rusiavimas ) {
00051     case 1:
00052         std::sort( studentai.begin( ), studentai.end( ), [ ] ( const Studentas& a, const Studentas& b
      ) {
00053             return a.vardas < b.vardas;
00054             } );
00055         break;
00056     case 2:
00057         std::sort( studentai.begin( ), studentai.end( ), [ ] ( const Studentas& a, const Studentas& b
      ) {
00058             return a.pavarde < b.pavarde;
00059             } );
00060         break;
00061     case 3:
00062         std::sort( studentai.begin( ), studentai.end( ), [ ] ( const Studentas& a, const Studentas& b
      ) {
00063             double ga = skaiciuotiGalutini( skaiciuotiVidurki( a.nd, a.n ), a.egzaminas );
00064             double gb = skaiciuotiGalutini( skaiciuotiVidurki( b.nd, b.n ), b.egzaminas );
00065             return ga > gb;
00066             } );
00067         break;
00068     case 4:
00069         std::sort( studentai.begin( ), studentai.end( ), [ ] ( const Studentas& a, const Studentas& b
      ) {
00070             double ga = skaiciuotiGalutini( skaiciuotiMediana( a.nd, a.n ), a.egzaminas );
00071             double gb = skaiciuotiGalutini( skaiciuotiMediana( b.nd, b.n ), b.egzaminas );
00072             return ga > gb;
00073             } );
00074         break;
00075     }
00076 }
```

7.15 ConsoleApplication1/studentas_utils.h File Reference

```
#include "studentas.h"
#include <vector>
```

Include dependency graph for studentas_utils.h: This graph shows which files directly or indirectly include this file:

Functions

- void generuotiPazymius (Studentas &s, int n)
- void generuotiVarda (Studentas &s, int indeksas)
- int pasirinktiRusiavima ()
- void rusiuotiStudentus (std::vector< Studentas > &studentai, int rusiavimas)

7.15.1 Function Documentation

7.15.1.1 generuotiPazymius()

```
void generuotiPazymius (
            Studentas & s,
            int n)
```

Definition at line 8 of file studentas_utils.cpp.

7.15.1.2 generuotiVarda()

```
void generuotiVarda (
            Studentas & s,
            int indeksas)
```

Definition at line 19 of file studentas_utils.cpp.

7.15.1.3 pasirinktiRusiavima()

```
int pasirinktiRusiavima ()
```

Definition at line 29 of file studentas_utils.cpp.

7.15.1.4 rusiuotiStudentus()

```
void rusiuotiStudentus (
            std::vector< Studentas > & studentai,
            int rusiavimas)
```

Definition at line 48 of file studentas_utils.cpp.

7.16 studentas_utils.h

Go to the documentation of this file.

```
00001 #ifndef STUDENTAS_UTILS_H
00002 #define STUDENTAS_UTILS_H
00003
00004 #include "studentas.h"
00005 #include <vector>
00006
00007 void generuotiPazymius( Studentas& s, int n );
00008 void generuotiVarda( Studentas& s, int indeksas );
00009 int pasirinktiRusiavima( );
00010 void rusiuotiStudentus( std::vector<Studentas>& studentai, int rusiavimas );
00011
00012
00013 #endif
```

7.17 ConsoleApplication1/studentu_io.cpp File Reference

Functions

- boolskaitytiSveika(int&reiksme,intmin_val,intmax_val)
- template<typenameStream>voidspausdintiIStream(Stream&out,conststd::vector<Studentas>&studentai,boolmediana)
- voidspausdintiRezultatus(std::vector<Studentas>&studentai,intm,boolmediana)
- while(!skaitytiSveika(n,1,100))
- while(true)
- if(!stream.is_open())throwFailoKlaida("Nepavykoatidarytifailokursiokai.txt.")
- if(!std::getline(stream,line))returnout
- while(header_stream>>column)cols.push_back(column)
- while(std::getline(stream,line))

Variables

- std=std::chrono::high_resolution_clock::now()
- stint&n
- m=0
- returnstudentai
- autostart=std::chrono::high_resolution_clock::now()

7.17.1 Function Documentation

7.17.1.1 a()

```
boolskaitytiSveika(
            int&reiksme,
            intmin_val,
            intmax_val)
```

Definition at line 12 of file studentu_io.cpp.

7.17.1.2 e() [1/4]

```
while (
            ! skaitytiSveik a n, 1, 1 0 0)
```

Definition at line 95 of file studentu_io.cpp.

7.17.1.3 e() [2/4]

```
while (
            header_stream,
            column)
```

7.17.1.4 e() [3/4]

```
while (
            std::getlin estream, line)
```

Definition at line 166 of file studentu_io.cpp.

7.17.1.5 e() [4/4]

```
while (
            true)
```

Definition at line 102 of file studentu_io.cpp.

7.17.1.6 f() [1/2]

```
if (
            ! std::getlin es tream, line)
```

7.17.1.7 f() [2/2]

```
if (
            ! stream. is_ope n())
```

7.17.1.8 m()

```
template< typename Stream > voidspausdintiIStream (
            Stream &out,
            conststd::vector< Studentas > &studenta i,
            boolmediana)
```

Definition at line 28 of file studentu_io.cpp.

7.17.1.9 s()

```
void spausdinti Rezultatus (
            std ::vector< Studentas > &studentai,
            int m,
            bool mediana)
```

Definition at line 62 of file studentu_io.cpp.

7.17.2 Variable Documentation

7.17.2.1 d

```
std = std : : chrono : : high_resolution_clock : : now (
)
```

Definition at line 91 of file studentu_io.cpp.

7.17.2.2 i

```
return studentai
```

Definition at line 139 of file studentu_io.cpp.

7.17.2.3 m

```
m = 0
```

Definition at line 99 of file studentu_io.cpp.

7.17.2.4 n

```
stint& n
```

Initial value:

```
{
        std::cout << "Kiek namu darbu? "
```

Definition at line 91 of file studentu_io.cpp.

7.17.2.5 t

```
return out = std : : chrono : : high_resolution_clock
: : now ( )
```

Definition at line 148 of file studentu_io.cpp.

7.18 studentu_io.cpp

Go to the documentation of this file.

```
00001 #include "studentu_io.h"
00002 #include "studentas_utils.h"
00003 #include "skaiciavimas.h"
00004 #include "exceptions.h"
00005 #include <iostream>
00006 #include <fstream>
00007 #include <sstream>
00008 #include <iomanip>
00009 #include <limits>
00010 #include <chrono>
00011
00012 bool skaitytiSveika( int& reiksme, int min_val, int max_val )
00013 {
00014         if ( !( std::cin >> reiksme ) )
00015         {
00016                 std::cin.clear( );
00017                 std::cin.ignore( std::numeric_limits<std::streamsize>::max( ), '\n' );
00018                 return false;
00019         }
00020
00021         if ( reiksme < min_val || reiksme > max_val )
00022                 return false;
00023
00024         return true;
00025 }
00026
00027 template <typename Stream>
00028 void spausdintiIStream( Stream& out, const std::vector<Studentas>& studentai, bool mediana )
00029 {
00030         out << "\n" << std::string( 70, '-' ) << "\n";
00031
00032         out << std::left << std::setw( 15 ) << "Pavarde"
00033                 << std::setw( 15 ) << "Vardas"
00034                 << std::setw( 20 ) << "Galutinis (Vid.)";
00035
00036         if ( mediana ) out << std::setw( 20 ) << "Galutinis (Med.)";
00037
00038         out << "\n" << std::string( 70, '-' ) << "\n";
00039
00040         for ( const auto& s : studentai )
00041         {
00042                 double vid = skaiciuotiGalutiniVidurki( s.nd, s.n ), s.egzaminas );
00043
00044                 out << std::left << std::setw( 15 ) << s.pavarde
00045                         << std::setw( 15 ) << s.vardas
00046                         << std::fixed << std::setprecision( 2 )
00047                         << std::setw( 20 ) << vid;
00048
00049                 if ( mediana )
00050                 {
00051                         double med = skaiciuotiGalutiniMediana( s.nd, s.n ), s.egzaminas );
00052                         out << std::setw( 20 ) << med;
00053                 }
00054
00055                 out << "\n";
00056         }
00057
00058         out << std::string( 70, '-' ) << "\n";
00059 }
00060
00061
00062 void spausdintiRezultatus( std::vector<Studentas>& studentai, int m, bool mediana )
00063 {
00064         int rusiavimas = pasirinktiRusiavima( );
00065         rusiuotiStudentus( studentai, rusiavimas );
00066
00067         int outputPasirinkimas;
00068         std::cout << "\nIsvedimo vieta:\n 1 - Ekranas\n 2 - Failas\nPasirinkimas: ";
00069
00070         while ( !skaitytiSveika( outputPasirinkimas, 1, 2 ) ) {
```

```
00071             std::cout << "Neteisinga reiksme. Pasirinkite 1 arba 2: ";
00072         }
00073
00074         if (outputPasirinkimas == 1)
00075         {
00076             spausdintiIStream(std::cout, studentai, mediana);
00077         }
00078         else
00079         {
00080             std::ofstream stream("rezultatai.txt");
00081             if (!stream.is_open())
00082             {
00083                 throw FailoKlaida("Nepavyko atidaryti failo rezultatai.txt rasymui.");
00084             }
00085
00086             spausdintiIStream(stream, studentai, mediana);
00087             std::cout << "Rezultatai issaugoti faile: rezultatai.txt\n";
00088         }
00089 }
00090
00091 std::vector<Studentas> ivestiRankiniu(int& m, int& n)
00092 {
00093         std::cout << "Kiek namu darbu? ";
00094
00095         while (!skaitytiSveika(n, 1, 100)) {
00096             std::cout << "Neteisinga reiksme. Iveskite 1-100: ";
00097         }
00098
00099         m = 0;
00100         std::vector<Studentas> studentai;
00101
00102         while (true)
00103         {
00104             Studentas studentas;
00105
00106             std::cout << "\n--- Studentas #" << (m + 1) << " (arba iveskite 'baigti') ---\n";
00107             std::cout << "Vardas: ";
00108
00109             std::cin >> studentas.vardas;
00110
00111             if (studentas.vardas == "baigti")
00112                 break;
00113
00114             std::cout << "Pavarde: ";
00115             std::cin >> studentas.pavarde;
00116
00117             studentas.n = n;
00118             studentas.nd.resize(n);
00119
00120             for (int j = 0; j < n; j++)
00121             {
00122                 std::cout << "  ND" << (j + 1) << " balas (1-10): ";
00123                 while (!skaitytiSveika(studentas.nd[j], 1, 10)) {
00124                     std::cout << "  Neteisinga reiksme (1-10): ";
00125                 }
00126             }
00127
00128             std::cout << "Egzamino balas (1-10): ";
00129
00130             while (!skaitytiSveika(studentas.egzaminas, 1, 10)) {
00131                 std::cout << "  Neteisinga reiksme (1-10): ";
00132             }
00133
00134             studentai.push_back(studentas);
00135
00136             m++;
00137         }
00138
00139         return studentai;
00140 }
00141
00142 std::vector<Studentas> nuskaitytiStudentus()
00143 {
00144         std::ifstream stream("kursiokai.txt");
```

```
00145         if ( !stream.is_open() )
00146             throw FailoKlaida("Nepavyko atidaryti failo kursiokai.txt.");
00147
00148         auto start = std::chrono::high_resolution_clock::now();
00149
00150         std::vector<Studentas> out;
00151
00152         std::string line;
00153         if ( !std::getline(stream, line) )
00154             return out;
00155
00156         std::stringstream header_stream(line);
00157
00158         std::string column;
00159         std::vector<std::string> cols;
00160
00161         while ( header_stream >> column )
00162             cols.push_back(column);
00163
00164         size_t nd_count = cols.size() >= 3 ? cols.size() - 3 : 0;
00165
00166         while ( std::getline(stream, line) )
00167         {
00168             if ( line.empty() )
00169                 continue;
00170
00171             std::stringstream ss(line);
00172
00173             Studentas studentas;
00174             studentas.n = static_cast<int>(nd_count);
00175             studentas.nd.resize(nd_count);
00176
00177             if ( !(ss >> studentas.vardas >> studentas.pavarde) )
00178                 throw DuomenuKlaida("Nepavyko nuskaityti vardo arba pavardes: " + line);
00179
00180             for ( size_t i = 0; i < nd_count; i++ )
00181             {
00182                 if ( !(ss >> studentas.nd[i]) )
00183                     throw DuomenuKlaida("Nepavyko nuskaityti namu darbu pazymio eiluteje: " + line);
00184             }
00185
00186             if ( !(ss >> studentas.egzaminas) )
00187                 throw DuomenuKlaida("Nepavyko nuskaityti egzamino balo eiluteje: " + line);
00188
00189             out.push_back(studentas);
00190         }
00191
00192         auto end = std::chrono::high_resolution_clock::now();
00193         auto elapsed = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);
00194         std::cout << "Nuskaityti " << out.size() << " studentu is per " << std::fixed << std::setprecision(3) << elapsed.count() << " ms\n";
00195
00196         return out;
00197 }
```

7.19 ConsoleApplication1/studentu_io.h File Reference

This graph shows which files directly or indirectly include this file:

Functions

- bool skaityti Sveika (int &reiksme, int min_val, int max_val)
- void spausdinti Rezultatus (std::vector< Studentas > &studentai, int m, bool mediana)

Variables

- std
- stint& n

7.19.1 Function Documentation

7.19.1.1 a()

```
bool skaityti Sveika (
            int &reiksme,
            int min_val,
            int max_val)
```

Definition at line 12 of file studentu_io.cpp.

7.19.1.2 s()

```
void spausdinti Rezultatus (
            std::vector< Studentas > &studentai,
            int m,
            bool mediana)
```

Definition at line 62 of file studentu_io.cpp.

7.19.2 Variable Documentation

7.19.2.1 d

```
std
```

Definition at line 12 of file studentu_io.h.

7.19.2.2 n

```
stint& n
```

Definition at line 12 of file studentu_io.h.

7.20 studentu_io.h

Go to the documentation of this file.

```
00001 #ifndef STUDENTU_IO_H
00002 #define STUDENTU_IO_H
00003
00004 #include "studentas.h"
00005 #include <vector>
00006 #include <iostream>
00007
00008 bool skaityti Sveika( int& reiksme, int min_val, int max_val );
00009
00010 void spausdinti Rezultatus( std::vector<Studentas>& studentai, int m, bool mediana );
00011
00012 std::vector<Studentas> ivesti Rankiniu( int& m, int& n );
00013 std::vector<Studentas> nuskaityti Studentus( );
00014
00015 #endif
```

7.21 ConsoleApplication1/testavimas.cpp File Reference

```
#include "testavimas.h"
#include "skaiciavimas.h"
#include "studentas_utils.h"
#include "exceptions.h"
#include "studentu_io.h"
#include "vector_stl.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <iomanip>
#include <chrono>
#include <cstdlib>
#include <string>
#include <algorithm>
#include <vector>
#include <list>
#include <deque>
```

Include dependency graph for testavimas.cpp:

Functions

- void testuotiKlase ()
- void generuotiFaila (const std::string &failoVardas, int irasu_sk, int nd_kiekis)
- std::vector< Studentas > nuskaitytiIsFailo (const std::string &failoVardas)
- void isvestiKategorijaIFaila (const std::string &failoVardas, const std::vector< Studentas > &studentai, bool mediana)
- void tyrimas1_failuKurimas ()
- void tyrimasKonteineriu (bool mediana)
- void tyrimasStrategiju (bool mediana)
- void tyrimasVectorPushBack ()
- void tyrimasVectorReallocations ()
- void tyrimasVectorStudentai (bool mediana)

7.21.1 Function Documentation

7.21.1.1 generuotiFaila()

```
void generuotiFaila (
            const std::string & failoVardas,
            int irasu_sk,
            int nd_kiekis)
```

Definition at line 183 of file testavimas.cpp.

7.21.1.2 isvestiKategorijaIFaila()

```
void isvestiKategorijaIFaila (
            const std::string & failoVardas,
            const std::vector< Studentas > & studentai,
            bool mediana)
```

Definition at line 227 of file testavimas.cpp.

7.21.1.3 nuskaitytiIsFailo()

```
std::vector< Studentas > nuskaitytiIsFailo (
            const std::string & failoVardas)
```

Definition at line 222 of file testavimas.cpp.

7.21.1.4 testuotiKlase()

```
void testuotiKlase ()
```

Definition at line 40 of file testavimas.cpp.

7.21.1.5 tyrimas1_failuKurimas()

```
void tyrimas1_failuKurimas ()
```

Definition at line 263 of file testavimas.cpp.

7.21.1.6 tyrimasKonteineriu()

```
void tyrimasKonteineriu (
            bool mediana)
```

Definition at line 314 of file testavimas.cpp.

7.21.1.7 tyrimasStrategiju()

```
void tyrimasStrategiju (
            bool mediana)
```

Definition at line 386 of file testavimas.cpp.

7.21.1.8 tyrimasVectorPushBack()

```
void tyrimasVectorPushBack ()
```

Definition at line 483 of file testavimas.cpp.

7.21.1.9 tyrimasVectorReallocations()

```
void tyrimasVectorReallocations ()
```

Definition at line 539 of file testavimas.cpp.

7.21.1.10 tyrimasVectorStudentai()

```
void tyrimasVectorStudentai (
            bool mediana)
```

Definition at line 593 of file testavimas.cpp.

7.22 testavimas.cpp

Go to the documentation of this file.

```
00001 #include "testavimas.h"
00002 #include "skaiciavimas.h"
00003 #include "studentas_utils.h"
00004 #include "exceptions.h"
00005 #include "studentu_io.h"
00006 #include "vector_stl.h"
00007
00008 #include <iostream>
00009 #include <fstream>
00010 #include <sstream>
00011 #include <iomanip>
00012 #include <chrono>
00013 #include <cstdlib>
00014 #include <string>
00015 #include <algorithm>
00016 #include <vector>
00017 #include <list>
00018 #include <deque>
00019
00020 static int g_ok   = 0;
00021 static int g_fail = 0;
00022
00023 static void spausdintiRezultata( const std::string& testas, bool rezultatas )
00024 {
00025     std::cout « "  " « ( rezultatas ? "[OK]  " : "[FAIL]" ) « " " « testas « "\n";
00026     if ( rezultatas ) ++g_ok; else ++g_fail;
00027 }
00028
00029 static Studentas sukurtiTestoStudenta()
00030 {
00031     Studentas s;
00032     s.vardas    = "Jonas";
00033     s.pavarde   = "Jonaitis";
00034     s.n         = 3;
00035     s.nd        = { 8, 9, 7 };
00036     s.egzaminas = 10;
00037     return s;
00038 }
00039
00040 void testuotiKlase()
00041 {
00042     std::cout « "\n======== Studentas klases testai ========\n\n";
00043
00044     std::cout « "0. Zmogus abstrakti klase:\n";
00045     spausdintiRezultata( "std::is_abstract<Zmogus>::value == true",
00046                          std::is_abstract<Zmogus>::value == true );
00047     spausdintiRezultata( "std::is_abstract<Studentas>::value == false",
00048                          std::is_abstract<Studentas>::value == false );
00049     spausdintiRezultata( "Studentas yra isvestine is Zmogus",
00050                          std::is_base_of<Zmogus, Studentas>::value == true );
00051
00052     std::cout « "\n1. Default konstruktorius:\n";
00053     {
00054         Studentas s;
00055         spausdintiRezultata( "vardas == \"\"",    s.vardas    == "" );
00056         spausdintiRezultata( "pavarde == \"\"",   s.pavarde   == "" );
00057         spausdintiRezultata( "n == 0",            s.n         == 0  );
00058         spausdintiRezultata( "egzaminas == 0",    s.egzaminas == 0  );
00059         spausdintiRezultata( "nd.empty()",        s.nd.empty()      );
00060     }
00061
00062     std::cout « "\n2. Kopijavimo konstruktorius:\n";
00063     {
00064         Studentas s1 = sukurtiTestoStudenta();
00065         Studentas s2( s1 );
00066
00067         spausdintiRezultata( "s2.vardas == \"Jonas\"",      s2.vardas    == "Jonas"    );
```

```
00068         spausdintiRezultata( "s2.pavarde == \"Jonaitis\"",  s2.pavarde   == "Jonaitis" );
00069         spausdintiRezultata( "s2.n == 3",                   s2.n         == 3          );
00070         spausdintiRezultata( "s2.egzaminas == 10",          s2.egzaminas == 10         );
00071         spausdintiRezultata( "s2.nd == {8,9,7}",            s2.nd        == std::vector<int>{ 8, 9, 7 }
      );
00072
00073         s2.vardas    = "Petras";
00074         s2.nd[ 0 ]   = 1;
00075         spausdintiRezultata( "originalas nepakito (vardas)", s1.vardas  == "Jonas" );
00076         spausdintiRezultata( "originalas nepakito (nd[0])",  s1.nd[ 0 ] == 8       );
00077     }
00078
00079     std::cout « "\n3. Kopijavimo priskyrimas (operator=):\n";
00080     {
00081         Studentas s1 = sukurtiTestoStudenta();
00082         Studentas s2;
00083         s2 = s1;
00084
00085         spausdintiRezultata( "s2.vardas == \"Jonas\"",      s2.vardas    == "Jonas"    );
00086         spausdintiRezultata( "s2.n == 3",                   s2.n         == 3          );
00087         spausdintiRezultata( "s2.egzaminas == 10",          s2.egzaminas == 10         );
00088         spausdintiRezultata( "s2.nd == {8,9,7}",            s2.nd        == std::vector<int>{ 8, 9, 7 }
      );
00089
00090         s2.pavarde = "Kazlauskas";
00091         spausdintiRezultata( "originalas nepakito (pavarde)", s1.pavarde == "Jonaitis" );
00092
00093         s1 = s1;
00094         spausdintiRezultata( "savipriskyrimass saugus",     s1.vardas == "Jonas" );
00095     }
00096
00097     std::cout « "\n4. Perkėlimo konstruktorius (move ctor):\n";
00098     {
00099         Studentas s1 = sukurtiTestoStudenta();
00100         Studentas s2( std::move( s1 ) );
00101
00102         spausdintiRezultata( "s2.vardas == \"Jonas\"",      s2.vardas    == "Jonas"    );
00103         spausdintiRezultata( "s2.n == 3",                   s2.n         == 3          );
00104         spausdintiRezultata( "s2.egzaminas == 10",          s2.egzaminas == 10         );
00105         spausdintiRezultata( "s2.nd == {8,9,7}",            s2.nd        == std::vector<int>{ 8, 9, 7 }
      );
00106         spausdintiRezultata( "s1.n == 0 (saltinis istusejo)", s1.n         == 0 );
00107         spausdintiRezultata( "s1.egzaminas == 0",             s1.egzaminas == 0 );
00108     }
00109
00110     std::cout « "\n5. Perkėlimo priskyrimas (move operator=):\n";
00111     {
00112         Studentas s1 = sukurtiTestoStudenta();
00113         Studentas s2;
00114         s2 = std::move( s1 );
00115
00116         spausdintiRezultata( "s2.vardas == \"Jonas\"",      s2.vardas    == "Jonas"    );
00117         spausdintiRezultata( "s2.n == 3",                   s2.n         == 3          );
00118         spausdintiRezultata( "s2.egzaminas == 10",          s2.egzaminas == 10         );
00119         spausdintiRezultata( "s2.nd == {8,9,7}",            s2.nd        == std::vector<int>{ 8, 9, 7 }
      );
00120         spausdintiRezultata( "s1.n == 0 (saltinis istusejo)", s1.n         == 0 );
00121         spausdintiRezultata( "s1.egzaminas == 0",             s1.egzaminas == 0 );
00122
00123         s2 = std::move( s2 );
00124         spausdintiRezultata( "saviperkėlimas saugus", s2.n == 3 );
00125     }
00126
00127     std::cout « "\n6. operator« (isvedimas):\n";
00128     {
00129         Studentas s = sukurtiTestoStudenta();
00130         std::ostringstream oss;
00131         oss « s;
00132         std::string rezultatas = oss.str();
00133         std::string laukiamas  = "Jonas Jonaitis 3 8 9 7 10";
00134         spausdintiRezultata( "operator« ekrane: \"" + rezultatas + "\"",
00135                              rezultatas == laukiamas );
00136
00137         std::ofstream failas( "test_studentas.txt" );
00138         failas « s;
00139         failas.close();
00140         spausdintiRezultata( "operator« iraše i faila", true );
00141     }
00142
00143     std::cout « "\n7. operator» (ivedimas):\n";
00144     {
00145         std::istringstream iss( "Ona Kazlauskiene 2 6 8 9" );
00146         Studentas s;
00147         iss » s;
00148         spausdintiRezultata( "vardas == \"Ona\"",           s.vardas    == "Ona"          );
00149         spausdintiRezultata( "pavarde == \"Kazlauskiene\"", s.pavarde  == "Kazlauskiene" );
00150         spausdintiRezultata( "n == 2",                      s.n         == 2              );
```

```
00151         spausdintiRezultata( "nd == {6,8}",                s.nd        == std::vector<int>{ 6, 8 } );
00152         spausdintiRezultata( "egzaminas == 9",             s.egzaminas == 9              );
00153
00154         std::ifstream failas( "test_studentas.txt" );
00155         Studentas sIsFailo;
00156         failas » sIsFailo;
00157         failas.close();
00158         spausdintiRezultata( "operator» is failo: vardas == \"Jonas\"",
00159                              sIsFailo.vardas == "Jonas" );
00160         spausdintiRezultata( "operator» is failo: n == 3",
00161                              sIsFailo.n == 3 );
00162     }
00163
00164     std::cout « "\n8. Destruktorius:\n";
00165     {
00166         {
00167             Studentas s = sukurtiTestoStudenta();
00168         }
00169         spausdintiRezultata( "destruktorius iskviestas be klaidu (scope pabaiga)", true );
00170     }
00171
00172     std::cout « "\n========================================\n";
00173     std::cout « "Rezultatai: " « g_ok « " OK, " « g_fail « " FAIL\n";
00174     if ( g_fail == 0 )
00175         std::cout « "Visi testai praejo sekmingai!\n\n";
00176     else
00177         std::cout « "Kai kurie testai nepraejo!\n\n";
00178
00179     g_ok = 0;
00180     g_fail = 0;
00181 }
00182
00183 void generuotiFaila( const std::string& failoVardas, int irasu_sk, int nd_kiekis )
00184 {
00185     std::ofstream out( failoVardas );
00186     if ( !out.is_open( ) )
00187         throw FailoKlaida( "Nepavyko sukurti failo: " + failoVardas );
00188
00189     out « std::left
00190         « std::setw( 25 ) « "Vardas"
00191         « std::setw( 27 ) « "Pavarde";
00192
00193     std::stringstream a2;
00194
00195     for ( int i = 1; i <= nd_kiekis; i++ )
00196         a2 « std::setw( 10 ) « ( "ND" + std::to_string( i ) );
00197
00198     out « a2.str( );
00199
00200     out « std::setw( 10 ) « "Egz." « "\n";
00201
00202     for ( int i = 1; i <= irasu_sk; i++ )
00203     {
00204         std::stringstream a3;
00205
00206         a3 « std::left
00207             « std::setw( 25 ) « ( "Vardas" + std::to_string( i ) )
00208             « std::setw( 27 ) « ( "Pavarde" + std::to_string( i ) )
00209             « std::right;
00210
00211         for ( int j = 0; j < nd_kiekis; j++ )
00212             a3 « std::setw( 10 ) « ( rand( ) % 10 + 1 );
00213
00214         a3 « std::setw( 10 ) « ( rand( ) % 10 + 1 ) « "\n";
00215
00216         out « a3.str( );
00217     }
00218
00219     out.close( );
00220 }
00221
00222 std::vector<Studentas> nuskaitytiIsFailo( const std::string& failoVardas )
00223 {
00224     return nuskaitytiIsFailoT<std::vector<Studentas>>( failoVardas );
00225 }
00226
00227 void isvestiKategorijaIFaila( const std::string& failoVardas,
00228     const std::vector<Studentas>& studentai, bool mediana )
00229 {
00230     std::ofstream out( failoVardas );
00231     if ( !out.is_open( ) )
00232         throw FailoKlaida( "Nepavyko sukurti failo: " + failoVardas );
00233
00234     out « std::left
00235         « std::setw( 20 ) « "Pavarde"
00236         « std::setw( 20 ) « "Vardas"
00237         « std::setw( 20 ) « "Galutinis" « "\n";
```

```
00238     out « std::string( 60, '-' ) « "\n";
00239
00240     for ( const auto& s : studentai )
00241     {
00242         double galutinis;
00243
00244         if ( mediana )
00245             galutinis = skaiciuotiGalutini( skaiciuotiMediana( s.nd, s.n ), s.egzaminas );
00246         else
00247             galutinis = skaiciuotiGalutini( skaiciuotiVidurki( s.nd, s.n ), s.egzaminas );
00248
00249         std::stringstream ss;
00250
00251         ss « std::left
00252             « std::setw( 20 ) « s.pavarde
00253             « std::setw( 20 ) « s.vardas
00254             « std::fixed « std::setprecision( 2 )
00255             « std::setw( 20 ) « galutinis « "\n";
00256
00257         out « ss.str( );
00258     }
00259
00260     out.close( );
00261 }
00262
00263 void tyrimas1_failuKurimas( )
00264 {
00265     const int dydziai[] = { 1000, 10000, 100000, 1000000, 10000000 };
00266     const std::string pavadinimai[] = {
00267         "studentai_1000.txt",
00268         "studentai_10000.txt",
00269         "studentai_100000.txt",
00270         "studentai_1000000.txt",
00271         "studentai_10000000.txt"
00272     };
00273     const int nd_kiekis = 15;
00274     const int bandymu_sk = 3;
00275
00276     std::cout « "\n================ 1 TYRIMAS: Failu kurimas ================\n\n";
00277
00278     std::cout « std::left
00279         « std::setw( 18 ) « "Irasu sk."
00280         « std::setw( 18 ) « "1 bandymas (s)"
00281         « std::setw( 18 ) « "2 bandymas (s)"
00282         « std::setw( 18 ) « "3 bandymas (s)"
00283         « std::setw( 18 ) « "Vidurkis (s)" « "\n";
00284     std::cout « std::string( 90, '-' ) « "\n";
00285
00286     for ( int i = 0; i < 5; i++ )
00287     {
00288         double laikai[bandymu_sk] = { };
00289         double suma = 0.0;
00290
00291         for ( int b = 0; b < bandymu_sk; b++ )
00292         {
00293             std::remove( pavadinimai[i].c_str( ) );
00294
00295             auto start = std::chrono::high_resolution_clock::now( );
00296             generuotiFaila( pavadinimai[i], dydziai[i], nd_kiekis );
00297             auto end = std::chrono::high_resolution_clock::now( );
00298
00299             laikai[b] = std::chrono::duration<double>( end - start ).count( );
00300             suma += laikai[b];
00301         }
00302
00303         std::cout « std::left « std::setw( 18 ) « dydziai[i]
00304             « std::fixed « std::setprecision( 5 )
00305             « std::setw( 18 ) « laikai[0]
00306             « std::setw( 18 ) « laikai[1]
00307             « std::setw( 18 ) « laikai[2]
00308             « std::setw( 18 ) « ( suma / bandymu_sk ) « "\n";
00309     }
00310
00311     std::cout « "\nFailai sugeneruoti ir issaugoti.\n";
00312 }
00313
00314 void tyrimasKonteineriu( bool mediana )
00315 {
00316     const int dydziai[] = { 1000, 10000, 100000, 1000000, 10000000 };
00317     const std::string pavadinimai[] = {
00318         "studentai_1000.txt",
00319         "studentai_10000.txt",
00320         "studentai_100000.txt",
00321         "studentai_1000000.txt",
00322         "studentai_10000000.txt"
00323     };
00324     const int bandymu_sk = 3;
```

```
00325
00326     std::cout « "\n================ 2 TYRIMAS: Konteineriu palyginimas ================\n";
00327     std::cout « "Kiekvienas matavimas atliktas " « bandymu_sk
00328               « " kartus, pateikiamas vidurkis.\n\n";
00329
00330     std::cout « std::left
00331         « std::setw( 12 ) « "Irasu sk."
00332         « std::setw( 14 ) « "Konteineris"
00333         « std::setw( 18 ) « "Nuskaitymas(s)"
00334         « std::setw( 18 ) « "Rusiavimas(s)"
00335         « std::setw( 18 ) « "Skaidymas(s)"
00336         « "\n";
00337     std::cout « std::string( 80, '-' ) « "\n";
00338
00339     for ( int i = 0; i < 5; i++ )
00340     {
00341         {
00342             std::ifstream test( pavadinimai[i] );
00343             if ( !test.is_open( ) )
00344             {
00345                 std::cout « std::left « std::setw( 12 ) « dydziai[i]
00346                     « "Failas nerastas! Pirma paleiskite 1 tyrima.\n";
00347                 continue;
00348             }
00349         }
00350
00351         double nusk, rus, skaid;
00352
00353         benchmarkKonteineris<std::vector<Studentas>>(
00354             pavadinimai[i], mediana, bandymu_sk, nusk, rus, skaid );
00355         std::cout « std::left « std::setw( 12 ) « dydziai[i]
00356             « std::setw( 14 ) « "vector"
00357             « std::fixed « std::setprecision( 5 )
00358             « std::setw( 18 ) « nusk
00359             « std::setw( 18 ) « rus
00360             « std::setw( 18 ) « skaid « "\n";
00361
00362         benchmarkKonteineris<std::list<Studentas>>(
00363             pavadinimai[i], mediana, bandymu_sk, nusk, rus, skaid );
00364         std::cout « std::left « std::setw( 12 ) « ""
00365             « std::setw( 14 ) « "list"
00366             « std::fixed « std::setprecision( 5 )
00367             « std::setw( 18 ) « nusk
00368             « std::setw( 18 ) « rus
00369             « std::setw( 18 ) « skaid « "\n";
00370
00371         benchmarkKonteineris<std::deque<Studentas>>(
00372             pavadinimai[i], mediana, bandymu_sk, nusk, rus, skaid );
00373         std::cout « std::left « std::setw( 12 ) « ""
00374             « std::setw( 14 ) « "deque"
00375             « std::fixed « std::setprecision( 5 )
00376             « std::setw( 18 ) « nusk
00377             « std::setw( 18 ) « rus
00378             « std::setw( 18 ) « skaid « "\n";
00379
00380         std::cout « std::string( 80, '-' ) « "\n";
00381     }
00382
00383     std::cout « "\nTyrimas baigtas.\n";
00384 }
00385
00386 void tyrimasStrategiju( bool mediana )
00387 {
00388     const int dydziai[] = { 1000, 10000, 100000, 1000000, 10000000 };
00389     const std::string pavadinimai[] = {
00390         "studentai_1000.txt",
00391         "studentai_10000.txt",
00392         "studentai_100000.txt",
00393         "studentai_1000000.txt",
00394         //"studentai_10000000.txt"
00395     };
00396
00397     const int bandymu_sk = 1;
00398
00399     std::cout « "\n================ 3 TYRIMAS: Strategiju palyginimas ================\n";
00400     std::cout « "Kiekvienas matavimas atliktas " « bandymu_sk
00401               « " kartus, pateikiamas vidurkis.\n";
00402     std::cout « "Matuojamas TIK skaidymo i grupes laikas (be nuskaitymo ir rusiavimo).\n";
00403
00404     const char* konteineriai[] = { "std::vector", "std::list", "std::deque" };
00405
00406     for ( int k = 0; k < 3; k++ )
00407     {
00408         std::cout « "\n--- " « konteineriai[k] « " ---\n\n";
00409         std::cout « std::left
00410             « std::setw( 15 ) « "Irasu sk."
00411             « std::setw( 20 ) « "1 strategija(s)"
```

```
00412             « std::setw( 20 ) « "2 strategija(s)"
00413             « std::setw( 20 ) « "3 strategija(s)"
00414             « "\n";
00415         std::cout « std::string( 75, '-' ) « "\n";
00416 
00417         for ( int i = 0; i < sizeof( pavadinimai) / sizeof( pavadinimai[0]); i++ )
00418         {
00419             {
00420                 std::ifstream test( pavadinimai[i] );
00421                 if ( !test.is_open( ) )
00422                 {
00423                     std::cout « std::left « std::setw( 15 ) « dydziai[i]
00424                         « "Failas nerastas!\n";
00425                     continue;
00426                 }
00427             }
00428 
00429             double s1, s2, s3;
00430 
00431             if ( k == 0 )
00432                 benchmarkStrategijos<std::vector<Studentas>>(
00433                     pavadinimai[i], mediana, bandymu_sk, s1, s2, s3 );
00434             else if ( k == 1 )
00435                 benchmarkStrategijos<std::list<Studentas>>(
00436                     pavadinimai[i], mediana, bandymu_sk, s1, s2, s3 );
00437             else
00438                 benchmarkStrategijos<std::deque<Studentas>>(
00439                     pavadinimai[i], mediana, bandymu_sk, s1, s2, s3 );
00440 
00441             std::cout « std::left « std::setw( 15 ) « dydziai[i]
00442                 « std::fixed « std::setprecision( 5 )
00443                 « std::setw( 20 ) « s1
00444                 « std::setw( 20 ) « s2
00445                 « std::setw( 20 ) « s3 « "\n";
00446         }
00447     }
00448 
00449     std::cout « "\nTyrimas baigtas.\n";
00450 
00451     std::cout « "\n--- Kietiakiai/vargsiukai issaugojimas i failus ---\n";
00452 
00453     bool rastasVienas = false;
00454     for ( int i = 0; i < 5; i++ )
00455     {
00456         std::ifstream test( pavadinimai[i] );
00457         if ( !test.is_open( ) )
00458             continue;
00459         test.close( );
00460 
00461         rastasVienas = true;
00462         std::string sk = std::to_string( dydziai[i] );
00463 
00464         std::vector<Studentas> base = nuskaitytiIsFailoT<std::vector<Studentas>>( pavadinimai[i] );
00465         rusiuotiPagalGalutini( base, mediana );
00466 
00467         std::vector<Studentas> data = base;
00468         std::vector<Studentas> kiet, varg;
00469         strategija1( data, kiet, varg, mediana );
00470 
00471         isvestiKategorijaIFaila( "kietiakiai_" + sk + ".txt", kiet, mediana );
00472         isvestiKategorijaIFaila( "vargsiukai_"  + sk + ".txt", varg, mediana );
00473 
00474         std::cout « "  " « std::setw( 10 ) « std::left « sk « " irasu  ->  "
00475                   « "kietiakiai_" « sk « ".txt (" « kiet.size( ) « "),  "
00476                   « "vargsiukai_"  « sk « ".txt (" « varg.size( ) « ")\n";
00477     }
00478 
00479     if ( !rastasVienas )
00480         std::cout « "  Nera failu - pirma paleiskite 1 tyrima.\n";
00481 }
00482 
00483 void tyrimasVectorPushBack( )
00484 {
00485     const size_t dydziai[] = { 10000, 100000, 1000000, 10000000, 100000000 };
00486     const int n = sizeof( dydziai ) / sizeof( dydziai[0] );
00487 
00488     std::cout « "\n================ V3.0 TYRIMAS: push_back greitis (std::vector vs Vector)
    ================\n";
00489     std::cout « "Matuojama tuscio konteinerio uzpildymas push_back metodu.\n\n";
00490 
00491     std::cout « std::left
00492         « std::setw( 15 ) « "Elementu sk."
00493         « std::setw( 20 ) « "std::vector (s)"
00494         « std::setw( 20 ) « "Vector (s)"
00495         « std::setw( 15 ) « "Lyginimas"
00496         « "\n";
00497     std::cout « std::string( 70, '-' ) « "\n";
```

```
00498
00499     for ( int i = 0; i < n; i++ )
00500     {
00501         size_t sz = dydziai[i];
00502
00503         double t_std = 0.0;
00504         {
00505             auto t1 = std::chrono::high_resolution_clock::now( );
00506
00507             std::vector<int> v1;
00508             for ( size_t k = 1; k <= sz; k++ )
00509                 v1.push_back( static_cast<int>( k ) );
00510
00511             auto t2 = std::chrono::high_resolution_clock::now( );
00512             t_std = std::chrono::duration<double>( t2 - t1 ).count( );
00513         }
00514
00515         double t_my = 0.0;
00516         {
00517             auto t1 = std::chrono::high_resolution_clock::now( );
00518
00519             Vector<int> v2;
00520             for ( size_t k = 1; k <= sz; k++ )
00521                 v2.push_back( static_cast<int>( k ) );
00522
00523             auto t2 = std::chrono::high_resolution_clock::now( );
00524             t_my = std::chrono::duration<double>( t2 - t1 ).count( );
00525         }
00526
00527         double rel = ( t_std > 0.0 ) ? ( t_my / t_std ) : 0.0;
00528
00529         std::cout « std::left « std::setw( 15 ) « sz
00530             « std::fixed « std::setprecision( 5 )
00531             « std::setw( 20 ) « t_std
00532             « std::setw( 20 ) « t_my
00533             « std::setprecision( 2 ) « "x" « rel « "\n";
00534     }
00535
00536     std::cout « "\nTyrimas baigtas.\n";
00537 }
00538
00539 void tyrimasVectorReallocations( )
00540 {
00541     const size_t SZ = 100000000;
00542
00543     std::cout « "\n================ V3.0 TYRIMAS: Atminties perskirstymai (push_back iki 100M)
    ================\n\n";
00544
00545     std::cout « "Pildomi std::vector ir Vector iki " « SZ « " elementu...\n\n";
00546
00547     size_t std_realloc = 0;
00548     size_t std_final_cap = 0;
00549     {
00550         std::vector<int> v1;
00551         size_t prev_cap = 0;
00552
00553         for ( size_t k = 1; k <= SZ; k++ )
00554         {
00555             v1.push_back( static_cast<int>( k ) );
00556             if ( v1.capacity( ) != prev_cap )
00557             {
00558                 std_realloc++;
00559                 prev_cap = v1.capacity( );
00560             }
00561         }
00562
00563         std_final_cap = v1.capacity( );
00564     }
00565
00566     Vector<int> v2;
00567     for ( size_t k = 1; k <= SZ; k++ )
00568         v2.push_back( static_cast<int>( k ) );
00569
00570     std::cout « std::left
00571         « std::setw( 18 ) « "Konteineris"
00572         « std::setw( 18 ) « "Final size"
00573         « std::setw( 18 ) « "Final capacity"
00574         « std::setw( 18 ) « "Perskirstymai"
00575         « "\n";
00576     std::cout « std::string( 70, '-' ) « "\n";
00577
00578     std::cout « std::left
00579         « std::setw( 18 ) « "std::vector<int>"
00580         « std::setw( 18 ) « SZ
00581         « std::setw( 18 ) « std_final_cap
00582         « std::setw( 18 ) « std_realloc
00583         « "\n";
```

```
00584
00585     std::cout « std::left
00586         « std::setw( 18 ) « "Vector<int>"
00587         « std::setw( 18 ) « v2.size( )
00588         « std::setw( 18 ) « v2.capacity( )
00589         « std::setw( 18 ) « v2.realloc_count( )
00590         « "\n";
00591 }
00592
00593 void tyrimasVectorStudentai( bool mediana )
00594 {
00595     const int dydziai[] = { 1000, 10000, 100000, 1000000, 10000000 };
00596     const std::string pavadinimai[] = {
00597         "studentai_1000.txt",
00598         "studentai_10000.txt",
00599         "studentai_100000.txt",
00600         "studentai_1000000.txt",
00601         "studentai_10000000.txt"
00602     };
00603     const int N = sizeof( dydziai ) / sizeof( dydziai[0] );
00604
00605     std::cout « "\n================ V3.0 TYRIMAS: Studentu programa std::vector vs Vector
      ================\n";
00606     std::cout « "Matuojamas pilnas darbas: nuskaitymas + rusiavimas + skaidymas (1 strategija).\n\n";
00607
00608     std::cout « std::left
00609         « std::setw( 14 ) « "Irasu sk."
00610         « std::setw( 20 ) « "std::vector (s)"
00611         « std::setw( 20 ) « "Vector (s)"
00612         « "\n";
00613     std::cout « std::string( 60, '-' ) « "\n";
00614
00615     for ( int i = 0; i < N; i++ )
00616     {
00617         std::ifstream test( pavadinimai[i] );
00618         if ( !test.is_open( ) )
00619         {
00620             std::cout « std::left « std::setw( 14 ) « dydziai[i]
00621                 « "Failas nerastas! Pirma paleiskite 1 tyrima.\n";
00622             continue;
00623         }
00624         test.close( );
00625
00626         const int bandymu_sk = 3;
00627
00628         double suma_std = 0.0;
00629         double suma_my  = 0.0;
00630
00631         for ( int b = 0; b < bandymu_sk; b++ )
00632         {
00633             auto t1 = std::chrono::high_resolution_clock::now( );
00634             std::vector<Studentas> data1 = nuskaitytiIsFailoT<std::vector<Studentas>>( pavadinimai[i]
      );
00635
00636             rusiuotiPagalGalutini( data1, mediana );
00637
00638             std::vector<Studentas> kiet1, varg1;
00639             strategija1( data1, kiet1, varg1, mediana );
00640
00641             auto t2 = std::chrono::high_resolution_clock::now( );
00642             suma_std += std::chrono::duration<double>( t2 - t1 ).count( );
00643         }
00644
00645         for ( int b = 0; b < bandymu_sk; b++ )
00646         {
00647             auto t1 = std::chrono::high_resolution_clock::now( );
00648             Vector<Studentas> data1 = nuskaitytiIsFailoT<Vector<Studentas>>( pavadinimai [ i ] );
00649
00650             rusiuotiPagalGalutini( data1, mediana );
00651
00652             Vector<Studentas> kiet1, varg1;
00653             strategija1( data1, kiet1, varg1, mediana );
00654
00655             auto t2 = std::chrono::high_resolution_clock::now( );
00656             suma_my += std::chrono::duration<double>( t2 - t1 ).count( );
00657         }
00658
00659         std::cout « std::left « std::setw( 14 ) « dydziai[i]
00660             « std::fixed « std::setprecision( 5 )
00661             « std::setw( 20 ) « ( suma_std / bandymu_sk )
00662             « std::setw( 20 ) « ( suma_my / bandymu_sk )
00663             « "\n";
00664     }
00665
00666     std::cout « "\nTyrimas baigtas.\n";
00667 }
```

7.23 ConsoleApplication1/testavimas.h File Reference

```
#include "studentas.h"
#include "zmogus.h"
#include "skaiciavimas.h"
#include "exceptions.h"
#include <string>
#include <vector>
#include <list>
#include <deque>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <type_traits>
#include <chrono>
#include <iostream>
#include <iomanip>
#include <iterator>
#include <typeinfo>
```

Include dependency graph for testavimas.h: This graph shows which files directly or indirectly include this file:

Functions

- void generuotiFaila (const std::string &failoVardas, int irasu_sk, int nd_kiekis)
- void tyrimas1_failuKurimas ()
- void tyrimasKonteineriu (bool mediana)
- void tyrimasStrategiju (bool mediana)
- void testuotiKlase ()
- void tyrimasVectorPushBack ()
- void tyrimasVectorReallocations ()
- void tyrimasVectorStudentai (bool mediana)
- std::vector< Studentas > nuskaitytiIsFailo (const std::string &failoVardas)
- void isvestiKategorijaIFaila (const std::string &failoVardas, const std::vector< Studentas > &studentai, bool mediana)
- template<typename Container>
  Container nuskaitytiIsFailoT (const std::string &failoVardas)
- double apskaiciuotiGalutiniBala (const Studentas &s, bool mediana)
- template<typename Container>
  void rusiuotiPagalGalutini (Container &c, bool mediana)
- template<typename Container>
  void strategija1 (const Container &studentai, Container &kietiakiai, Container &vargsiukai, bool mediana)
- template<typename Container>
  void strategija2 (Container &studentai, Container &vargsiukai, bool mediana)
- template<typename Container>
  void strategija3 (Container &studentai, Container &vargsiukai, bool mediana)
- template<typename Container>
  void benchmarkKonteineris (const std::string &failas, bool mediana, int bandymu_sk, double &nusk_avg, double &rus_avg, double &skaid_avg)
- template<typename Container>
  void benchmarkStrategijos (const std::string &failas, bool mediana, int bandymu_sk, double &str1_avg, double &str2_avg, double &str3_avg)

7.23.1 Function Documentation

7.23.1.1 apskaiciuotiGalutiniBala()

```
double apskaiciuotiGalutiniBala (
            const Studentas & s,
            bool mediana)  [inline]
```

Definition at line 91 of file testavimas.h.

7.23.1.2 benchmarkKonteineris()

```
template<typename Container>
void benchmarkKonteineris (
            const std::string & failas,
            bool mediana,
            int bandymu_sk,
            double & nusk_avg,
            double & rus_avg,
            double & skaid_avg)
```

Definition at line 157 of file testavimas.h.

7.23.1.3 benchmarkStrategijos()

```
template<typename Container>
void benchmarkStrategijos (
            const std::string & failas,
            bool mediana,
            int bandymu_sk,
            double & str1_avg,
            double & str2_avg,
            double & str3_avg)
```

Definition at line 186 of file testavimas.h.

7.23.1.4 generuotiFaila()

```
void generuotiFaila (
            const std::string & failoVardas,
            int irasu_sk,
            int nd_kiekis)
```

Definition at line 183 of file testavimas.cpp.

7.23.1.5 isvestiKategorijaIFaila()

```
void isvestiKategorijaIFaila (
            const std::string & failoVardas,
            const std::vector< Studentas > & studentai,
            bool mediana)
```

Definition at line 227 of file testavimas.cpp.

7.23.1.6 nuskaitytiIsFailo()

```
std::vector< Studentas > nuskaitytiIsFailo (
            const std::string & failoVardas)
```

Definition at line 222 of file testavimas.cpp.

7.23.1.7 nuskaitytiIsFailoT()

```
template<typename Container>
Container nuskaitytiIsFailoT (
            const std::string & failoVardas)
```

Definition at line 39 of file testavimas.h.

7.23.1.8 rusiuotiPagalGalutini()

```
template<typename Container>
void rusiuotiPagalGalutini (
            Container & c,
            bool mediana)
```

Definition at line 100 of file testavimas.h.

7.23.1.9 strategija1()

```
template<typename Container>
void strategija1 (
            const Container & studentai,
            Container & kietiakiai,
            Container & vargsiukai,
            bool mediana)
```

Definition at line 113 of file testavimas.h.

7.23.1.10 strategija2()

```
template<typename Container>
void strategija2 (
            Container & studentai,
            Container & vargsiukai,
            bool mediana)
```

Definition at line 126 of file testavimas.h.

7.23.1.11 strategija3()

```
template<typename Container>
void strategija3 (
             Container & studentai,
             Container & vargsiukai,
             bool mediana)
```

Definition at line 136 of file testavimas.h.

7.23.1.12 testuotiKlase()

```
void testuotiKlase ()
```

Definition at line 40 of file testavimas.cpp.

7.23.1.13 tyrimas1_failuKurimas()

```
void tyrimas1_failuKurimas ()
```

Definition at line 263 of file testavimas.cpp.

7.23.1.14 tyrimasKonteineriu()

```
void tyrimasKonteineriu (
             bool mediana)
```

Definition at line 314 of file testavimas.cpp.

7.23.1.15 tyrimasStrategiju()

```
void tyrimasStrategiju (
             bool mediana)
```

Definition at line 386 of file testavimas.cpp.

7.23.1.16 tyrimasVectorPushBack()

```
void tyrimasVectorPushBack ()
```

Definition at line 483 of file testavimas.cpp.

7.23.1.17 tyrimasVectorReallocations()

```
void tyrimasVectorReallocations ()
```

Definition at line 539 of file testavimas.cpp.

7.23.1.18 tyrimasVectorStudentai()

```
void tyrimasVectorStudentai (
            bool mediana)
```

Definition at line 593 of file testavimas.cpp.

7.24 testavimas.h

Go to the documentation of this file.

```
00001 #ifndef TESTAVIMAS_H
00002 #define TESTAVIMAS_H
00003
00004 #include "studentas.h"
00005 #include "zmogus.h"
00006 #include "skaiciavimas.h"
00007 #include "exceptions.h"
00008
00009 #include <string>
00010 #include <vector>
00011 #include <list>
00012 #include <deque>
00013 #include <fstream>
00014 #include <sstream>
00015 #include <algorithm>
00016 #include <type_traits>
00017 #include <chrono>
00018 #include <iostream>
00019 #include <iomanip>
00020 #include <iterator>
00021 #include <typeinfo>
00022
00023 void generuotiFaila( const std::string& failoVardas, int irasu_sk, int nd_kiekis );
00024 void tyrimas1_failuKurimas( );
00025 void tyrimasKonteineriu( bool mediana );
00026 void tyrimasStrategiju( bool mediana );
00027 void testuotiKlase( );
00028
00029 void tyrimasVectorPushBack( );
00030 void tyrimasVectorReallocations( );
00031 void tyrimasVectorStudentai( bool mediana );
00032
00033 std::vector<Studentas> nuskaitytiIsFailo( const std::string& failoVardas );
00034
00035 void isvestiKategorijaIFaila( const std::string& failoVardas,
00036     const std::vector<Studentas>& studentai, bool mediana );
00037
00038 template<typename Container>
00039 Container nuskaitytiIsFailoT( const std::string& failoVardas )
00040 {
00041     std::ifstream stream( failoVardas );
00042     if ( !stream.is_open( ) )
00043         throw FailoKlaida( "Nepavyko atidaryti failo: " + failoVardas );
00044
00045     Container out;
00046     std::string line;
00047
00048     std::stringstream a2;
00049     a2 « stream.rdbuf( );
00050
00051     if ( !std::getline( a2, line ) )
00052         return out;
00053
00054     std::stringstream header_stream( line );
00055     std::string column;
00056     std::vector<std::string> cols;
00057
00058     while ( header_stream » column )
00059         cols.push_back( column );
00060
00061     size_t nd_count = cols.size( ) >= 3 ? cols.size( ) - 3 : 0;
00062
00063     while ( std::getline( a2, line ) )
00064     {
00065         if ( line.empty( ) )
00066             continue;
00067
```

```
00068         std::stringstream ss( line );
00069         Studentas s;
00070         s.n = static_cast<int>( nd_count );
00071         s.nd.resize( nd_count );
00072 
00073         if ( !( ss » s.vardas » s.pavarde ) )
00074             throw DuomenuKlaida( "Nepavyko nuskaityti: " + line );
00075 
00076         for ( size_t j = 0; j < nd_count; j++ )
00077         {
00078             if ( !( ss » s.nd[j] ) )
00079                 throw DuomenuKlaida( "Nepavyko nuskaityti ND: " + line );
00080         }
00081 
00082         if ( !( ss » s.egzaminas ) )
00083             throw DuomenuKlaida( "Nepavyko nuskaityti egzamino: " + line );
00084 
00085         out.push_back( s );
00086     }
00087 
00088     return out;
00089 }
00090 
00091 inline double apskaiciuotiGalutiniBala( const Studentas& s, bool mediana )
00092 {
00093     if ( mediana )
00094         return skaiciuotiGalutini( skaiciuotiMediana( s.nd, s.n ), s.egzaminas );
00095     else
00096         return skaiciuotiGalutini( skaiciuotiVidurki( s.nd, s.n ), s.egzaminas );
00097 }
00098 
00099 template<typename Container>
00100 void rusiuotiPagalGalutini( Container& c, bool mediana )
00101 {
00102     auto comp = [mediana]( const Studentas& a, const Studentas& b ) {
00103         return apskaiciuotiGalutiniBala( a, mediana ) > apskaiciuotiGalutiniBala( b, mediana );
00104     };
00105 
00106     if constexpr ( std::is_same_v<Container, std::list<Studentas» )
00107         c.sort( comp );
00108     else
00109         std::sort( c.begin( ), c.end( ), comp );
00110 }
00111 
00112 template<typename Container>
00113 void strategija1( const Container& studentai, Container& kietiakiai,
00114     Container& vargsiukai, bool mediana )
00115 {
00116     for ( const auto& s : studentai )
00117     {
00118         if ( apskaiciuotiGalutiniBala( s, mediana ) >= 5.0 )
00119             kietiakiai.push_back( s );
00120         else
00121             vargsiukai.push_back( s );
00122     }
00123 }
00124 
00125 template<typename Container>
00126 void strategija2( Container& studentai, Container& vargsiukai, bool mediana )
00127 {
00128     while ( apskaiciuotiGalutiniBala( studentai.back( ), mediana ) < 5.0 )
00129     {
00130         vargsiukai.push_back( studentai.back( ) );
00131         studentai.pop_back( );
00132     }
00133 }
00134 
00135 template<typename Container>
00136 void strategija3( Container& studentai, Container& vargsiukai, bool mediana )
00137 {
00138     auto it = std::stable_partition( studentai.begin( ), studentai.end( ),
00139         [mediana]( const Studentas& s ) {
00140             return apskaiciuotiGalutiniBala( s, mediana ) >= 5.0;
00141         } );
00142 
00143     if constexpr ( std::is_same_v<Container, std::list<Studentas» )
00144     {
00145         vargsiukai.splice( vargsiukai.end( ), studentai, it, studentai.end( ) );
00146     }
00147     else
00148     {
00149         vargsiukai.insert( vargsiukai.end( ),
00150             std::make_move_iterator( it ),
00151             std::make_move_iterator( studentai.end( ) ) );
00152         studentai.erase( it, studentai.end( ) );
00153     }
00154 }
```

```
00155
00156 template<typename Container>
00157 void benchmarkKonteineris( const std::string& failas, bool mediana, int bandymu_sk,
00158     double& nusk_avg, double& rus_avg, double& skaid_avg )
00159 {
00160     double nusk_suma = 0, rus_suma = 0, skaid_suma = 0;
00161
00162     for ( int b = 0; b < bandymu_sk; b++ )
00163     {
00164         auto t1 = std::chrono::high_resolution_clock::now( );
00165         Container data = nuskaitytiIsFailoT<Container>( failas );
00166         auto t2 = std::chrono::high_resolution_clock::now( );
00167
00168         rusiuotiPagalGalutini( data, mediana );
00169         auto t3 = std::chrono::high_resolution_clock::now( );
00170
00171         Container kiet, varg;
00172         strategija1( data, kiet, varg, mediana );
00173         auto t4 = std::chrono::high_resolution_clock::now( );
00174
00175         nusk_suma += std::chrono::duration<double>( t2 - t1 ).count( );
00176         rus_suma += std::chrono::duration<double>( t3 - t2 ).count( );
00177         skaid_suma += std::chrono::duration<double>( t4 - t3 ).count( );
00178     }
00179
00180     nusk_avg = nusk_suma / bandymu_sk;
00181     rus_avg = rus_suma / bandymu_sk;
00182     skaid_avg = skaid_suma / bandymu_sk;
00183 }
00184
00185 template<typename Container>
00186 void benchmarkStrategijos( const std::string& failas, bool mediana, int bandymu_sk,
00187     double& str1_avg, double& str2_avg, double& str3_avg )
00188 {
00189     Container original = nuskaitytiIsFailoT<Container>( failas );
00190     rusiuotiPagalGalutini( original, mediana );
00191
00192     double suma1 = 0, suma2 = 0, suma3 = 0;
00193
00194     for ( int b = 0; b < bandymu_sk; b++ )
00195     {
00196         {
00197             Container data = original;
00198             Container kiet, varg;
00199             auto t1 = std::chrono::high_resolution_clock::now( );
00200             strategija1( data, kiet, varg, mediana );
00201             auto t2 = std::chrono::high_resolution_clock::now( );
00202             suma1 += std::chrono::duration<double>( t2 - t1 ).count( );
00203         }
00204         {
00205             Container data = original;
00206             Container varg;
00207             auto t1 = std::chrono::high_resolution_clock::now( );
00208             strategija2( data, varg, mediana );
00209             auto t2 = std::chrono::high_resolution_clock::now( );
00210             suma2 += std::chrono::duration<double>( t2 - t1 ).count( );
00211         }
00212         {
00213             Container data = original;
00214             Container varg;
00215             auto t1 = std::chrono::high_resolution_clock::now( );
00216             strategija3( data, varg, mediana );
00217             auto t2 = std::chrono::high_resolution_clock::now( );
00218             suma3 += std::chrono::duration<double>( t2 - t1 ).count( );
00219         }
00220     }
00221
00222     str1_avg = suma1 / bandymu_sk;
00223     str2_avg = suma2 / bandymu_sk;
00224     str3_avg = suma3 / bandymu_sk;
00225 }
00226
00227 #endif
```

7.25 ConsoleApplication1/vector_stl.h File Reference

Sablono klase Vector<T>, atkartojanti std::vector funkcionaluma.

```
#include <stdexcept>
#include <limits>
```

```
#include <utility>
#include <initializer_list>
#include <iterator>
#include <algorithm>
```

Include dependency graph for vector_stl.h: This graph shows which files directly or indirectly include this file:

Data Structures

- class Vector< T >

  Dinaminis masyvas, atkartojantis std::vector elgesi.

7.25.1 Detailed Description

Sablono klase Vector<T>, atkartojanti std::vector funkcionaluma.

Definition in file vector_stl.h.

7.26 vector_stl.h

Go to the documentation of this file.

```
00001 #pragma once
00002
00003 #include <stdexcept>
00004 #include <limits>
00005 #include <utility>
00006 #include <initializer_list>
00007 #include <iterator>
00008 #include <algorithm>
00009
00014
00029 template<typename T>
00030 class Vector {
00031 private:
00032     T* data_ = nullptr;
00033     size_t size_ = 0;
00034     size_t capacity_ = 0;
00035     size_t realloc_count_ = 0;
00036
00037 public:
00038     using iterator = T*;
00039     using const_iterator = const T*;
00040
00041 public:
00043     Vector( ) : data_( nullptr ), size_( 0 ), capacity_( 0 ), realloc_count_( 0 ) { }
00044
00050     explicit Vector( size_t count, const T& value = T( ) ) : data_( nullptr ), size_( 0 ), capacity_(
      0 ), realloc_count_( 0 )
00051     {
00052         if ( count > 0 )
00053         {
00054             reserve( count );
00055
00056             for ( size_t i = 0; i < count; i++ )
00057                 data_ [ i ] = value;
00058
00059             size_ = count;
00060         }
00061     }
00062
00064     Vector( std::initializer_list<T> il ) : data_( nullptr ), size_( 0 ), capacity_( 0 ),
      realloc_count_( 0 )
00065     {
00066         reserve( il.size( ) );
00067
00068         for ( const auto& v : il )
00069             data_ [ size_++ ] = v;
00070     }
00071
```

```
00073     ~Vector( ) { delete[ ] data_; }
00074
00076     Vector( const Vector& other ) : data_( nullptr ), size_( other.size_ ), capacity_( other.capacity_
      ), realloc_count_( 0 )
00077     {
00078         if ( capacity_ > 0 )
00079         {
00080             data_ = new T [ capacity_ ];
00081
00082             for ( size_t i = 0; i < size_; i++ )
00083                 data_ [ i ] = other.data_ [ i ];
00084         }
00085     }
00086
00088     Vector& operator=( const Vector& other )
00089     {
00090         if ( this == &other ) return *this;
00091
00092         delete[ ] data_;
00093
00094         size_ = other.size_;
00095         capacity_ = other.capacity_;
00096         data_ = capacity_ > 0 ? new T [ capacity_ ] : nullptr;
00097
00098         for ( size_t i = 0; i < size_; i++ )
00099             data_ [ i ] = other.data_ [ i ];
00100
00101         return *this;
00102     }
00103
00105     Vector( Vector&& other ) noexcept : data_( other.data_ ), size_( other.size_ ), capacity_(
      other.capacity_ ), realloc_count_( other.realloc_count_ )
00106     {
00107         other.data_ = nullptr;
00108         other.size_ = 0;
00109         other.capacity_ = 0;
00110         other.realloc_count_ = 0;
00111     }
00112
00114     Vector& operator=( Vector&& other ) noexcept
00115     {
00116         if ( this == &other ) return *this;
00117
00118         delete[ ] data_;
00119
00120         data_ = other.data_;
00121         size_ = other.size_;
00122         capacity_ = other.capacity_;
00123         realloc_count_ = other.realloc_count_;
00124
00125         other.data_ = nullptr;
00126         other.size_ = 0;
00127         other.capacity_ = 0;
00128         other.realloc_count_ = 0;
00129
00130         return *this;
00131     }
00132
00133 public:
00135     bool operator==( const Vector& other ) const
00136     {
00137         if ( size_ != other.size_ )
00138             return false;
00139
00140         for ( size_t i = 0; i < size_; i++ )
00141             if ( data_ [ i ] != other.data_ [ i ] )
00142                 return false;
00143
00144         return true;
00145     }
00146
00148     bool operator<( const Vector& other ) const
00149     {
00150         for ( size_t i = 0; i < size_ && i < other.size_; i++ )
00151         {
00152             if ( data_ [ i ] < other.data_ [ i ] ) return true;
00153             if ( data_ [ i ] > other.data_ [ i ] ) return false;
00154         }
00155         return size_ < other.size_;
00156     }
00157
00158     bool operator!=( const Vector& other ) const { return !( *this == other ); }
00159     bool operator<=( const Vector& other ) const { return !( other < *this ); }
00160     bool operator>( const Vector& other )  const { return other < *this; }
00161     bool operator>=( const Vector& other ) const { return !( *this < other ); }
00162
00163 private:
```

```
00168     void internal_grow( size_t wanted = 0 )
00169     {
00170         size_t new_capacity = wanted == 0
00171             ? ( capacity_ == 0 ? 1 : capacity_ * 2 )
00172             : wanted;
00173 
00174         T* new_data = new T [ new_capacity ];
00175         for ( size_t i = 0; i < size_; i++ )
00176             new_data [ i ] = std::move( data_ [ i ] );
00177 
00178         delete[ ] data_;
00179         data_ = new_data;
00180         capacity_ = new_capacity;
00181 
00182         realloc_count_++;
00183     }
00184 
00185 public:
00187     T& operator[]( size_t index ) { return data_ [ index ]; }
00188     const T& operator[]( size_t index ) const { return data_ [ index ]; }
00189 
00191     T& at( size_t index )
00192     {
00193         if ( index >= size_ )
00194             throw std::out_of_range( "Vector::at index out of range" );
00195 
00196         return data_ [ index ];
00197     }
00198 
00199     const T& at( size_t index ) const
00200     {
00201         if ( index >= size_ )
00202             throw std::out_of_range( "Vector::at index out of range" );
00203 
00204         return data_ [ index ];
00205     }
00206 
00207     T& front( ) { return data_ [ 0 ]; }
00208     const T& front( ) const { return data_ [ 0 ]; }
00209 
00210     T& back( ) { return data_ [ size_ - 1 ]; }
00211     const T& back( )  const { return data_ [ size_ - 1 ]; }
00212 
00213     T* data( ) { return data_; }
00214     const T* data( ) const { return data_; }
00215 
00217     bool empty( ) const { return size_ == 0; }
00218 
00220     size_t size( ) const { return size_; }
00221 
00223     size_t capacity( ) const { return capacity_; }
00224 
00226     size_t max_size( ) const { return std::numeric_limits<size_t>::max( ) / sizeof( T ); }
00227 
00229     size_t realloc_count( ) const { return realloc_count_; }
00230 
00235     void reserve( size_t new_cap )
00236     {
00237         if ( new_cap <= capacity_ ) return;
00238         internal_grow( new_cap );
00239     }
00240 
00244     void shrink_to_fit( )
00245     {
00246         if ( size_ == capacity_ ) return;
00247 
00248         if ( size_ == 0 )
00249         {
00250             delete[ ] data_;
00251             data_ = nullptr;
00252 
00253             capacity_ = 0;
00254             realloc_count_++;
00255             return;
00256         }
00257 
00258         T* new_data = new T [ size_ ];
00259 
00260         for ( size_t i = 0; i < size_; i++ )
00261             new_data [ i ] = std::move( data_ [ i ] );
00262 
00263         delete[ ] data_;
00264 
00265         data_ = new_data;
00266         capacity_ = size_;
00267         realloc_count_++;
00268     }
```

```
00269
00270 public:
00271     iterator       begin( ) { return data_; }
00272     iterator       end( ) { return data_ + size_; }
00273
00274     const_iterator begin( )  const { return data_; }
00275     const_iterator end( )    const { return data_ + size_; }
00276
00277     const_iterator cbegin( ) const { return data_; }
00278     const_iterator cend( )   const { return data_ + size_; }
00279
00280 public:
00282     void push_back( const T& value )
00283     {
00284         if ( size_ == capacity_ )
00285             internal_grow( );
00286
00287         data_ [ size_++ ] = value;
00288     }
00289
00291     void push_back( T&& value )
00292     {
00293         if ( size_ == capacity_ )
00294             internal_grow( );
00295         data_ [ size_++ ] = std::move( value );
00296     }
00297
00299     void pop_back( )
00300     {
00301         if ( empty( ) )
00302             throw std::runtime_error( "Vector::pop_back on empty vector" );
00303
00304         size_--;
00305     }
00306
00308     void clear( ) { size_ = 0; }
00309
00314     iterator insert( const_iterator pos, const T& value )
00315     {
00316         size_t index = pos - data_;
00317         if ( index > size_ )
00318             throw std::out_of_range( "Vector::insert index out of range" );
00319
00320         if ( size_ == capacity_ )
00321             internal_grow( );
00322
00323         for ( size_t i = size_; i > index; i-- )
00324             data_ [ i ] = std::move( data_ [ i - 1 ] );
00325
00326         data_ [ index ] = value;
00327         size_++;
00328         return data_ + index;
00329     }
00330
00335     iterator erase( const_iterator pos )
00336     {
00337         size_t index = pos - data_;
00338         if ( index >= size_ )
00339             throw std::out_of_range( "Vector::erase index out of range" );
00340
00341         for ( size_t i = index; i + 1 < size_; i++ )
00342             data_ [ i ] = std::move( data_ [ i + 1 ] );
00343
00344         size_--;
00345         return data_ + index;
00346     }
00347
00351     template<typename ... Args>
00352     iterator emplace( const_iterator pos, Args&&... args )
00353     {
00354         size_t index = pos - data_;
00355         if ( index > size_ )
00356             throw std::out_of_range( "Vector::emplace index out of range" );
00357
00358         if ( size_ == capacity_ )
00359             internal_grow( );
00360
00361         for ( size_t i = size_; i > index; i-- )
00362             data_ [ i ] = std::move( data_ [ i - 1 ] );
00363
00364         data_ [ index ] = T( std::forward<Args>( args )... );
00365         size_++;
00366         return data_ + index;
00367     }
00368
00370     template<typename ... Args>
00371     void emplace_back( Args&&... args )
```

```
00372     {
00373         if ( size_ == capacity_ )
00374             internal_grow( );
00375 
00376         data_ [ size_ ] = T( std::forward<Args>( args )... );
00377         size_++;
00378     }
00379 
00383     void resize( size_t count, const T& value = T( ) )
00384     {
00385         if ( count == size_ ) return;
00386 
00387         if ( count < size_ )
00388         {
00389             size_ = count;
00390             return;
00391         }
00392 
00393         if ( count > capacity_ )
00394             internal_grow( count );
00395 
00396         for ( size_t i = size_; i < count; i++ )
00397             data_ [ i ] = value;
00398 
00399         size_ = count;
00400     }
00401 
00403     void swap( Vector& other ) noexcept
00404     {
00405         std::swap( data_, other.data_ );
00406         std::swap( size_, other.size_ );
00407         std::swap( capacity_, other.capacity_ );
00408         std::swap( realloc_count_, other.realloc_count_ );
00409     }
00410 };
```

7.27 ConsoleApplication1/zmogus.cpp File Reference

```
#include "zmogus.h"
```

Include dependency graph for zmogus.cpp:

7.28 zmogus.cpp

Go to the documentation of this file.

```
00001 #include "zmogus.h"
00002 
00003 Zmogus::Zmogus() : vardas( "" ), pavarde( "" )
00004 {
00005 }
00006 
00007 Zmogus::~Zmogus()
00008 {
00009 }
```

7.29 ConsoleApplication1/zmogus.h File Reference

```
#include <string>
#include <iostream>
```

Include dependency graph for zmogus.h: This graph shows which files directly or indirectly include this file:

Data Structures

- class Zmogus

7.30 zmogus.h

Go to the documentation of this file.

```
00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005 #include <iostream>
00006
00007 class Zmogus
00008 {
00009 public:
00010     std::string vardas;
00011     std::string pavarde;
00012
00013     Zmogus();
00014     virtual ~Zmogus() = 0;
00015 };
00016
00017 #endif
```

Index

~Studentas
    Studentas, 13
~Vector
    Vector< T >, 18
~Zmogus
    Zmogus, 28

a
    studentu_io.cpp, 41
    studentu_io.h, 47
apskaiciuotiGalutiniBala
    testavimas.h, 59
at
    Vector< T >, 19

back
    Vector< T >, 19
begin
    Vector< T >, 20
benchmarkKonteineris
    testavimas.h, 59
benchmarkStrategijos
    testavimas.h, 59

capacity
    Vector< T >, 20
capacity_
    Vector< T >, 27
cbegin
    Vector< T >, 20
cend
    Vector< T >, 20
clear
    Vector< T >, 20
ConsoleApplication1 Directory Reference, 9
ConsoleApplication1/exceptions.h, 29
ConsoleApplication1/main.cpp, 29, 30
ConsoleApplication1/skaiciavimas.cpp, 32, 33
ConsoleApplication1/skaiciavimas.h, 33, 34
ConsoleApplication1/studentas.cpp, 34, 35
ConsoleApplication1/studentas.h, 36, 37
ConsoleApplication1/studentas_utils.cpp, 37, 39
ConsoleApplication1/studentas_utils.h, 40, 41
ConsoleApplication1/studentu_io.cpp, 41, 44
ConsoleApplication1/studentu_io.h, 46, 47
ConsoleApplication1/testavimas.cpp, 48, 50
ConsoleApplication1/testavimas.h, 58, 62
ConsoleApplication1/vector_stl.h, 64, 65
ConsoleApplication1/zmogus.cpp, 69
ConsoleApplication1/zmogus.h, 69, 70
const_iterator
    Vector< T >, 17

d
    studentu_io.cpp, 43
    studentu_io.h, 47
data
    Vector< T >, 21
data_
    Vector< T >, 27
DuomenuKlaida, 11
    DuomenuKlaida, 11

e
    studentu_io.cpp, 41, 42
egzaminas
    Studentas, 14
emplace
    Vector< T >, 21
emplace_back
    Vector< T >, 21
empty
    Vector< T >, 21
end
    Vector< T >, 22
erase
    Vector< T >, 22

f
    studentu_io.cpp, 42
FailoKlaida, 12
    FailoKlaida, 12
front
    Vector< T >, 22

generuotiFaila
    testavimas.cpp, 48
    testavimas.h, 59
generuotiPazymius
    studentas_utils.cpp, 38
    studentas_utils.h, 40
generuotiVarda
    studentas_utils.cpp, 38
    studentas_utils.h, 40

i
    studentu_io.cpp, 43
insert
    Vector< T >, 22
internal_grow
    Vector< T >, 23

isvestiKategorijaIFaila
testavimas.cpp, 48
testavimas.h, 59
iterator
Vector< T >, 17

m
studentu_io.cpp, 42, 43
main
main.cpp, 30
main.cpp
main, 30
max_size
Vector< T >, 23

n
Studentas, 14
studentu_io.cpp, 43
studentu_io.h, 47
nd
Studentas, 14
nuskaitytiIsFailo
testavimas.cpp, 48
testavimas.h, 59
nuskaitytiIsFailoT
testavimas.h, 60

operator!=
Vector< T >, 23
operator<
Vector< T >, 23
operator<<
studentas.cpp, 35
studentas.h, 37
operator<=
Vector< T >, 24
operator>
Vector< T >, 24
operator>>
studentas.cpp, 35
studentas.h, 37
operator>=
Vector< T >, 25
operator=
Studentas, 14
Vector< T >, 24
operator==
Vector< T >, 24
operator[]
Vector< T >, 25

pasirinktiRusiavima
studentas_utils.cpp, 38
studentas_utils.h, 40
pavarde
Zmogus, 28
pop_back
Vector< T >, 25
push_back
Vector< T >, 25

realloc_count
Vector< T >, 26
realloc_count_
Vector< T >, 27
reserve
Vector< T >, 26
resize
Vector< T >, 26
rusiuotiPagalGalutini
testavimas.h, 60
rusiuotiStudentus
studentas_utils.cpp, 38
studentas_utils.h, 40

s
studentu_io.cpp, 42
studentu_io.h, 47
shrink_to_fit
Vector< T >, 26
size
Vector< T >, 26
size_
Vector< T >, 27
skaiciavimas.cpp
skaiciuotiGalutini, 32
skaiciuotiMediana, 32
skaiciuotiVidurki, 33
skaiciavimas.h
skaiciuotiGalutini, 34
skaiciuotiMediana, 34
skaiciuotiVidurki, 34
skaiciuotiGalutini
skaiciavimas.cpp, 32
skaiciavimas.h, 34
skaiciuotiMediana
skaiciavimas.cpp, 32
skaiciavimas.h, 34
skaiciuotiVidurki
skaiciavimas.cpp, 33
skaiciavimas.h, 34
strategija1
testavimas.h, 60
strategija2
testavimas.h, 60
strategija3
testavimas.h, 60
Studentas, 12
~Studentas, 13
egzaminas, 14
n, 14
nd, 14
operator=, 14
Studentas, 13
studentas.cpp
operator<<, 35
operator>>, 35
studentas.h

operator<<, 37
operator>>, 37
studentas_utils.cpp
generuotiPazymius, 38
generuotiVarda, 38
pasirinktiRusiavima, 38
rusiuotiStudentus, 38
studentas_utils.h
generuotiPazymius, 40
generuotiVarda, 40
pasirinktiRusiavima, 40
rusiuotiStudentus, 40
studentu_io.cpp
a, 41
d, 43
e, 41, 42
f, 42
i, 43
m, 42, 43
n, 43
s, 42
t, 43
studentu_io.h
a, 47
d, 47
n, 47
s, 47
swap
Vector< T >, 27

t
studentu_io.cpp, 43
testavimas.cpp
generuotiFaila, 48
isvestiKategorijaIFaila, 48
nuskaitytiIsFailo, 48
testuotiKlase, 49
tyrimas1_failuKurimas, 49
tyrimasKonteineriu, 49
tyrimasStrategiju, 49
tyrimasVectorPushBack, 49
tyrimasVectorReallocations, 49
tyrimasVectorStudentai, 49
testavimas.h
apskaiciuotiGalutiniBala, 59
benchmarkKonteineris, 59
benchmarkStrategijos, 59
generuotiFaila, 59
isvestiKategorijaIFaila, 59
nuskaitytiIsFailo, 59
nuskaitytiIsFailoT, 60
rusiuotiPagalGalutini, 60
strategija1, 60
strategija2, 60
strategija3, 60
testuotiKlase, 61
tyrimas1_failuKurimas, 61
tyrimasKonteineriu, 61
tyrimasStrategiju, 61
tyrimasVectorPushBack, 61
tyrimasVectorReallocations, 61
tyrimasVectorStudentai, 61
testuotiKlase
testavimas.cpp, 49
testavimas.h, 61
tyrimas1_failuKurimas
testavimas.cpp, 49
testavimas.h, 61
tyrimasKonteineriu
testavimas.cpp, 49
testavimas.h, 61
tyrimasStrategiju
testavimas.cpp, 49
testavimas.h, 61
tyrimasVectorPushBack
testavimas.cpp, 49
testavimas.h, 61
tyrimasVectorReallocations
testavimas.cpp, 49
testavimas.h, 61
tyrimasVectorStudentai
testavimas.cpp, 49
testavimas.h, 61

vardas
Zmogus, 28
Vector
Vector< T >, 18, 19
Vector< T >, 15
~Vector, 18
at, 19
back, 19
begin, 20
capacity, 20
capacity_, 27
cbegin, 20
cend, 20
clear, 20
const_iterator, 17
data, 21
data_, 27
emplace, 21
emplace_back, 21
empty, 21
end, 22
erase, 22
front, 22
insert, 22
internal_grow, 23
iterator, 17
max_size, 23
operator!=, 23
operator<, 23
operator<=, 24
operator>, 24
operator>=, 25
operator=, 24
operator==, 24

operator[], 25
pop_back, 25
push_back, 25
realloc_count, 26
realloc_count_, 27
reserve, 26
resize, 26
shrink_to_fit, 26
size, 26
size_, 27
swap, 27
Vector, 18, 19

Zmogus, 28
~Zmogus, 28
pavarde, 28
vardas, 28
Zmogus, 28